

정보통신공학과 졸업논문

2D/3D 디지털 트윈 기반
대학 셔틀버스 통합 운행
정보 시스템의 설계 및 구현

순천향대학교 프로토타입

Design and Implementation of an Integrated University Shuttle Bus
Information System Based on a 2D/3D Digital Twin

지도교수 박 동 규

20214157 권 재 원

박 소 정 · 손 성 윤 · 김 동 하

정 영 선 · 강 민 영

2026년 7월

순 천 향 대 학 교

정보통신공학과

2D/3D 디지털 트윈 기반 대학 셔틀버스 통합 운행 정보 시스템의 설계 및 구현

순천향대학교 프로토타입

이 논문을 정보통신공학과 졸업논문으로 제출함

2026년 7월

20214157 권재원

박소정 · 손성윤 · 김동하

정영선 · 강민영

순천향대학교 정보통신공학과

졸업논문 승인

권재원 외 5인의 졸업논문을 심사하고
정보통신공학과 졸업논문으로 인정함

지도교수 박 동 규 (인)

2026년 7월

순 천 향 대 학 교
정보통신공학과

목 차

국문초록.....	5
제1장 서론.....	1
1.1 연구 배경.....	1
1.2 문제 정의.....	2
1.3 연구 목적과 연구질문.....	3
1.4 연구 범위와 기여.....	3
1.5 논문의 구성.....	4
제2장 관련 연구 및 기술 배경.....	5
2.1 대학 셔틀 추적과 운영 관리.....	5
2.2 GPS 기반 도착시간 예측.....	6
2.3 스마트 캠퍼스와 이동 디지털 트윈.....	7
2.4 PWA와 웹 기반 3D 기술.....	7
2.5 인증과 데이터 접근 보안.....	8
2.6 선행연구의 공백과 본 연구의 위치.....	10
제3장 연구 방법 및 요구사항.....	11
3.1 설계과학 연구 절차.....	11
3.2 이해관계자와 운영 제약.....	12
3.3 기능 요구사항.....	12
3.4 비기능 요구사항.....	13
3.5 평가 설계.....	14
제4장 시스템 설계.....	15
4.1 전체 아키텍처.....	15
4.2 데이터 모델.....	16
4.3 노선과 운행 규칙 모델.....	18
4.4 경로 편집과 동기화.....	19
4.5 2D·3D 표현 설계.....	20
4.6 인증·권한 및 보안 설계.....	20
제5장 구현.....	22
5.1 개발 환경과 기술 스택.....	22
5.2 사용자 화면.....	23
5.3 2D 지도와 마커 구현.....	24
5.4 3D 캠퍼스와 차량 시뮬레이션.....	25
5.5 관리자와 기사 화면.....	26
5.6 공지, 알림과 PWA.....	27
5.7 개발 협업과 형상관리.....	28
제6장 평가 결과.....	29
6.1 평가 환경.....	29
6.2 저장소 규모와 정적 품질.....	29
6.3 알고리즘 시나리오 시험.....	30
6.4 데이터베이스와 API 권한 시험.....	31
6.5 빌드 산출물과 브라우저 성능.....	32
6.6 보안 검토 결과.....	33
6.7 연구질문별 결과 요약.....	34

제7장 논의.....	36
7.1 통합 정보 구조의 의미.....	36
7.2 2D와 3D의 상호보완성.....	36
7.3 ETA의 해석과 확장.....	37
7.4 기사 조작 제약과 운영 구조.....	38
7.5 보안과 개인정보 논의.....	38
7.6 연구의 한계.....	39
7.7 후속 연구와 구현 로드맵.....	40
제8장 결론.....	41
참고문헌.....	43
부록 A. 상세 시험 항목.....	45
부록 B. 팀 역할과 개발 일정.....	46
부록 C. 연구윤리 및 AI 활용 고지.....	46
Abstract.....	48

표 목 차

표 3-1. 설계과학 단계와 UNIBUS 연구 활동.....	11
표 3-2. 이해관계자별 주요 요구사항.....	12
표 4-1. 셔틀 운행 규칙의 데이터 표현.....	18
표 5-1. 주요 기술 스택과 역할.....	22
표 6-1. 평가 환경과 도구.....	29
표 6-2. 저장소 규모와 검사 결과.....	30
표 6-3. 알고리즘 시나리오 시험 결과.....	30
표 6-4. DB와 비인증 API 시험 결과.....	32
표 6-5. Lighthouse 5회 반복 측정 중앙값.....	33
표 6-6. 잔여 보안 위험과 개선 우선순위.....	34
표 A-1. 알고리즘 시험 21개 항목.....	45
표 B-1. 팀 구성과 담당 역할.....	46
표 B-2. 개발 및 연구 일정.....	46

그림 목 차

그림 4-1. UNIBUS 시스템 아키텍처.....	15
그림 4-2. UNIBUS 핵심 데이터 모델.....	17
그림 4-3. 노선 편집과 실시간 운행 정보 흐름.....	19
그림 5-1. 사용자 중심 단일 스크롤 온보딩 화면.....	23
그림 5-2. UNIBUS 사용자 모바일 주요 화면.....	24
그림 5-3. 관리자 대시보드와 노선 관리 화면.....	26
그림 6-1. 프로토타입 정량 평가 요약.....	33

국문초록

대학 셔틀버스는 학내순환, 철도역 연계, 광역 통학처럼 운행 목적과 시간표 기준이 서로 다른 노선을 함께 제공한다. 그러나 노선과 정류장, 차량 상태, 공지, 사용자 역할이 여러 매체에 분산되면 학생은 현재 이용 가능한 교통수단을 한눈에 파악하기 어렵고, 관리자는 변경된 경로나 배차 규칙을 각 화면에 반복 반영해야 한다. 본 연구는 순천향대학교 교통 환경을 사례로 2차원 지도와 3차원 캠퍼스 시각화를 결합한 통합 셔틀버스 정보 시스템 UNIBUS를 설계하고 구현하였다.

연구는 설계과학 연구방법을 적용하여 문제 정의, 요구사항 도출, 인공물 설계·개발, 시나리오 시연, 정량 평가의 순서로 수행하였다. 시스템은 React와 Vite 기반 PWA, NAVER 지도 API, React Three Fiber, Supabase Edge Function, PostgreSQL과 Realtime으로 구성하였다. 학내순환은 10분 간격, 후문발 신창역 셔틀은 열차 출발 10분 전, 신창역발 셔틀은 열차 도착 약 5분 후라는 서로 다른 운행 규칙을 데이터 모델에 반영하였다. 또한 사용자, 관리자, 기사 역할을 분리하고 관리자가 지도에서 정류장과 경로 보정점을 편집하면 사용자 2D·3D 화면이 동일한 노선 데이터를 읽도록 연결하였다. 학교 측 문의에서 운전 중 기사의 직접 앱 조작이 현실적으로 어렵다는 제약이 확인되어, 기사 화면은 배차 확인과 운행 전후 상태 처리에 한정하고 위치 송신은 자동화를 전제로 설계하였다.

평가는 실제 운행 사용자 실험 대신 저장소 기반 정적 검사, 알고리즘 시나리오 시험, 데이터베이스와 API 권한 시험, 프로덕션 빌드 분석, Lighthouse 반복 측정으로 구성하였다. TypeScript와 ESLint 검사는 오류 없이 통과했고, 시간표·경로 이동·ETA 상태·시뮬레이션을 다룬 21개 시험이 모두 통과하였다. 로컬과 원격의 마이그레이션 18개가 일치했고 원격 데이터베이스 lint에서 스키마 오류가 발견되지 않았다. 비인증 사용자의 사용자 관리·기사 상태·노선 생성 요청은 401로 거부되었고, npm audit에서 알려진 의존성 취약점은 0건이었다. 공개 화면을 Lighthouse 기본 모바일 조건에서 각각 5회 측정한 결과 Performance 중앙값은 온보딩 79점, 독립 3D 화면 73점이었으며, 두 화면의

Accessibility와 Best Practices는 100점, TBT와 CLS는 0이었다. 반면 3D vendor 청크의 gzip 크기가 256.47 kB로 가장 컸고 LCP 중앙값은 4.654초로 측정되어 추가 최적화가 필요했다.

본 연구의 기여는 캠퍼스 내부 순환과 철도 연계 셔틀, 통학버스를 단일 노선·운행 모델로 통합하고, 관리자 편집 결과를 2D·3D 표현에 일관되게 연결한 프로토타입을 제시한 데 있다. 다만 실제 GPS 이력, 교통량, 승하차 정차시간이 없으므로 화면의 ETA는 경로 진행률과 설정 속도에 기반한 결정론적 추정이며 실제 도착 예측 정확도를 의미하지 않는다. 현장 사용자 평가와 기사 운행 시험도 수행하지 못했으므로 사용성, 운행 효율, 안전성은 후속 연구에서 검증해야 한다.

주요어: 대학 셔틀버스, 디지털 트윈, 실시간 위치, 도착시간 추정, 3D 캠퍼스, PWA, Supabase, 설계과학

제1장 서론

1.1 연구 배경

대학 캠퍼스의 교통 서비스는 일반 시내버스와 다른 운영 맥락을 가진다. 강의 시작과 종료 시각에 수요가 집중되고, 학내순환 차량은 짧은 구간을 반복 운행하며, 철도역 연계 셔틀은 열차 도착·출발 시각을 기준으로 대기과 출발을 결정한다. 광역 통학버스는 등교와 하교 중심의 정해진 시간표를 제공한다. 같은 학생이 하루 안에 세 유형을 연속해서 이용할 수 있지만, 노선 안내와 실시간 상태가 서로 다른 페이지나 공지에 분리되면 이동 결정을 내리는 데 필요한 정보가 단절된다.

순천향대학교 사례에서는 학내순환, 신창역과 후문을 왕복하는 셔틀, 지역별 통학버스가 존재한다. 학내순환은 약 10분 간격으로 출발하고, 후문에서 신창역으로 가는 차량은 열차 출발 시각의 10분 전에 출발한다. 신창역에서 후문으로 오는 차량은 열차 도착 전에 역 인근 정류장에서 대기하다가 도착 후 약 5분 동안 승객을 태우고 출발한다. 일부 차량은 후문 도착으로 운행을 마치고, 일부 차량은 학내순환을 한 차례 연계한 뒤 종료한다. 이 차이는 단순한 출발 시각 배열보다 운행 단계와 연계 노선을 함께 표현할 수 있는 데이터 구조를 요구한다.

위치 기반 셔틀 서비스에 관한 기존 연구는 GPS를 이용한 차량 추적과 도착시간 예측이 학생의 대기 계획을 지원할 수 있음을 보여준다 (Gonzales et al., 2024). 그러나 위치를 지도에 표시하는 기능과 신뢰할 수 있는 도착시간을 예측하는 기능은 같지 않다. 도착 예측 연구는 GPS 자료 정제, 노선 정합, 구간 속도, 정류장 정차시간, 과거와 실시간 상태를 함께 고려해 왔다 (Lin & Zeng, 1999; Cathey & Dailey, 2003; Sun et al., 2007). 따라서 시연용 경로 애니메이션을 운영형 예측으로 과장하지 않고 각 성속도 단계를 분리해 설계할 필요가 있다.

한편 디지털 트윈 연구는 현실 공간의 객체와 상태를 3차원 환경에 연결하여 복잡한 정보를 직관적으로 이해하도록 돕는다. 스마트 캠퍼스 연구는 건물 모델과 서버·데이터베이스·센서 정보를 결합한 3D 관리 환경을 제시했고 (Chen et al., 2024), 이동 디지털 트윈 연구는 차량과 보행자 같은 이동 객체의 물리-디지털 동기성과 표현 충실도를 주요 평가 요소로 다루었다 (Liu et al., 2023). 캠퍼스 셔틀에서는 정류장 이름만 나열하는 대신 건물·도로·고저차와 차량 진행 방향을 함께 보여주면 이용자가 노선의 공간 맥락을 이해하는 데 도움이 될 수 있다. 다만 정밀 측량과 실제 센서 동기화가 없는 모델은 운영형 디지털 트윈이 아니라 공간 기반 시각화 프로토타입으로 제한해 설명해야 한다.

1.2 문제 정의

본 연구가 해결하려는 문제는 네 가지로 정리된다. 첫째, 학내순환·신창역 셔틀·통학버스가 서로 다른 시간표 규칙과 정류장 구성을 가지지만 이를 하나의 서비스에서 일관되게 표현할 공통 모델이 필요하다. 둘째, 관리자가 수정한 정류장 좌표와 노선 경로가 사용자 2D 지도와 3D 화면에 같은 방식으로 반영되어야 한다. 셋째, 사용자·관리자·기사의 정보 요구와 쓰기 권한이 다르므로 화면과 API, 데이터베이스 권한을 함께 분리해야 한다. 넷째, 운전 중 기사 앱을 직접 조작하기 어렵다는 운영 제약을 고려해 자동 위치 송신과 관리자 사전 배정을 중심으로 설계해야 한다.

초기 개발 과정에서는 화면별 임시 좌표와 별도 경로 데이터가 함께 존재하여 관리자가 수정한 값이 일부 사용자 화면에 즉시 반영되지 않는 문제가 있었다. 2D 지도 마커의 픽셀 기준점과 지도 좌표를 혼동하면 확대·축소에 따라 정류장이 이동해 보였고, 3D 지형과 경로의 높이가 가까우면 깊이 버퍼 충돌로 선이 흔들리는 현상도 나타났다. 이러한 문제는 개별 화면의 CSS만으로 해결하기 어렵고, 노선 데이터의 단일 출처, 좌표 변환 규칙, 표현 계층 분리가 필요함을 보여준다.

1.3 연구 목적과 연구질문

본 연구의 목적은 순천향대학교 셔틀 운영 규칙을 반영한 통합 웹 시스템을 설계·구현하고, 현장 운행 시험이 제한된 조건에서 프로토타입의 기능적 완성성, 배포 성능, 기본 보안 상태를 재현 가능한 방법으로 평가하는 것이다. 이를 위해 다음 연구질문을 설정하였다.

RQ1. 학내순환, 신창역 연계 셔틀, 통학버스와 사용자·관리자·기사 역할을 하나의 데이터 및 서비스 구조로 어떻게 통합할 수 있는가?

RQ2. 경로 시뮬레이션, 시간표·ETA 계산, 2D·3D 시각화는 정의한 기능·성능·보안 기준을 어느 정도 충족하는가?

RQ3. 운전 중 기사의 직접 조작이 제한되는 운영 조건에서 어떤 관리자·차량 단말 중심 구조가 적합한가?

1.4 연구 범위와 기여

연구 범위는 웹 기반 프로토타입의 설계, 구현, 정적·시나리오 평가로 제한한다. 사용자 앱은 온보딩, 인증, 홈, 셔틀, 통학버스, 공지, 설정을 포함하고, 관리자 웹은 대시보드, 노선·정류장, 버스, 사용자, 공지, 알림, 3D 관리와 데모 시뮬레이션을 포함한다. 기사 화면은 배정 차량 선택, 운행 시작·종료, 위치 상태를 다룬다. PWA 설치와 백그라운드 Web Push도 구현 범위에 포함한다.

반면 실제 차량에 전용 GPS 단말을 장착하거나 기사에게 운전 중 입력을 요구하는 시험, 승객 대상 사용성 설문, 실제 도착시간 오차 측정, 통신 장애·배터리 소모·대규모 동시접속 시험은 제외한다. 학교 측에 문의한 결과 기사 직접 조작 방식은 현실적으로 적용하기 어렵다는 답변을 받은 것이 주요 이유다. 이 제약은 기능 부재로 숨기지 않고 연구 설계의 경계조건으로 사용한다.

본 연구의 주요 기여는 다음과 같다. 첫째, 세 종류의 캠퍼스 교통 서비스를 공통 노선·정류장·운행 상태 모델에 통합한다. 둘째, 관리자가 수정한 경로를 사용자 2D와 3D 화면이 공동으로 사용하도록 단일 데이터 흐름을 구성한다.

셋째, 열차 시각을 기준으로 출발 전 · 도착 후 오프셋과 연계 학내순환을 표현하는 운행 규칙을 데이터화한다. 넷째, 실제 현장 자료가 없는 상태에서도 저장소 · DB · API · 브라우저 측정으로 프로토타입의 구현 수준과 잔여 위험을 정량적으로 제시한다.

1.5 논문의 구성

제2장은 대학 셔틀 추적, 도착시간 예측, 디지털 트윈, PWA와 보안 관련 연구를 검토한다. 제3장은 설계과학 방법과 요구사항, 평가 계획을 설명한다. 제4장은 시스템 아키텍처, 데이터 모델, 운행 규칙과 권한 구조를 제시하고, 제5장은 프론트엔드 · 2D · 3D · 관리자 · 기사 · 알림 구현을 기술한다. 제6장은 정적 검사, 알고리즘 시험, DB · API 보안 시험, Lighthouse와 번들 측정 결과를 보고한다. 제7장은 연구질문별 해석, 실무적 함의, 한계와 후속 연구를 논의하며, 제8장에서 결론을 제시한다.

제2장 관련 연구 및 기술 배경

2.1 대학 셔틀 추적과 운영 관리

대학 셔틀 애플리케이션은 일반 대중교통 서비스보다 작은 운영 범위를 가지지만 학생과 운영자의 요구를 함께 다뤄야 한다. Gonzales 등(2024)은 대학의 종이 기반 예약·추적 절차를 모바일 애플리케이션으로 전환하고 GPS 위치, 학생 등록, 관리자 기능을 통합한 사례를 제시하였다. 이 연구는 실시간 위치 정보가 학생의 이동 계획을 지원하고 관리 기능과 결합될 수 있음을 보여준다. 그러나 예약이 핵심인 해당 사례와 달리 UNIBUS의 주된 문제는 여러 운행 유형의 노선과 시간표, 2D·3D 표현을 통합하는 데 있다. 따라서 예약 기능이나 다른 대학의 수용성 조사 결과를 그대로 적용하지 않았다.

캠퍼스 교통은 시간표 기반 서비스와 빈도 기반 서비스가 공존한다. 학내순환은 10분 간격이라는 빈도 규칙이 중요하고, 신창역 셔틀은 특정 열차 시각과의 시간 관계가 중요하다. GTFS Realtime 권고는 차량 위치와 운행 갱신이 변경 시 또는 최소 30초마다 새로 제공되고 차량 위치 자료가 90초보다 오래되지 않도록 관리할 것을 제안한다(MobilityData, n.d.). 이 기준은 데이터 형식 전체를 즉시 도입하지 않더라도 위치 신선도와 지속적인 식별자 관리의 운영 목표를 세우는 데 유용하다.

UNIBUS는 최신 위치가 45초를 초과하면 오래된 상태로 표시하도록 설계하였다. 이 값은 사용자에게 신선하지 않은 위치를 구분하는 내부 기준이며, 실제 차량 단말이 45초 이내에 반드시 송신한다는 의미는 아니다. 운영 단계에서는 송신 주기, 서버 수신 지연, 마지막 차량 시각을 각각 기록하고 피드 신뢰성을 별도로 모니터링해야 한다.

2.2 GPS 기반 도착시간 예측

Lin과 Zeng(1999)은 GPS 자료의 오류를 선별하고 노선을 표현한 뒤 운행 시간과 정차시간을 활용하는 도착 예측 알고리즘을 실험하였다. Dailey 등(2001)은 자동 차량 위치 자료와 과거 통계, 필터를 결합해 대규모 버스 도착 예측을 구현하였다. Cathey와 Dailey(2003)는 이를 Tracker, Filter, Predictor의 분리된 구조로 정리하였다. 이 흐름은 좌표 수집, 상태 보정, 예측을 서로 다른 책임으로 두어야 함을 시사한다.

Sun 등(2007)은 실시간 · 과거 구간속도와 시간 · 공간 변동, GIS 지도 정합을 결합하였다. Han 등(2020)은 위치 보정과 이동 · 정차 요인, 과거 · 실시간 자료를 LSTM에 입력하여 다중 정류장 도착을 예측하였다. 두 연구는 데이터가 축적될수록 복잡한 시공간 패턴을 활용할 수 있음을 보여주지만, 학습 자료가 없는 초기 시스템에 같은 방법을 적용하면 검증되지 않은 숫자를 제시할 위험이 있다.

GPS 좌표 자체도 도로와 노선에서 벗어날 수 있다. Newson과 Krumm(2009)은 잡음과 낮은 표본 빈도를 가진 GPS를 도로망에 정합하기 위해 은닉 마르코프 모델을 사용하였다. Elliott와 Lumley(2020)는 GTFS 기반 도로망과 GPS 차량 위치를 결합해 실시간 이동시간을 모델링하였다. 이러한 연구를 종합하면 실제 운영형 ETA는 최소한 경로 식별자, 지도 정합, 위치 신선도, 구간 진행률, 정차 · 대기 상태를 구분해야 한다.

현재 UNIBUS는 관리자가 저장한 경로의 누적 길이와 시뮬레이션 진행률, 설정 속도를 이용해 예상 도착시간을 계산한다. 이 값은 화면 상태와 경계조건을 검증하는 데 사용하며, 실제 교통을 예측하는 모델로 정의하지 않는다. 실제 GPS와 운행 이력이 축적되면 결정론적 기준선을 유지한 채 구간별 경험 속도, 정류장 정차 분포, 지도 정합, 통계 · 학습 모델을 단계적으로 비교할 수 있다.

2.3 스마트 캠퍼스와 이동 디지털 트윈

디지털 트윈은 물리 객체와 디지털 표현의 연결을 강조하지만 연구 분야에 따라 필요한 충실도와 동기화 수준이 다르다. Hu(2023)는 캠퍼스 환경 · 설비 · 구성원 · 교육 자원을 디지털화해 3D 시각화와 분석, 피드백을 제공하는 스마트 캠퍼스 플랫폼을 제안하였다. Chen 등(2024)은 3D 건물 모델, WebGIS, 서버, 데이터베이스와 IoT 정보를 연결하여 정적 화면이 아니라 사건 발생 시 객체 정보를 갱신하는 환경을 구현하였다.

이동 분야에서 Liu 등(2023)은 차량, 운전자, 보행자 같은 물리 이동 객체의 형상 표현과 물리-디지털 동기성을 다루었다. 이 연구는 단순 3D 장면과 이동 디지털 트윈 사이의 차이를 분명하게 한다. 3D 객체가 존재하는 것만으로 충분하지 않고, 실제 상태와의 연결 시점, 지연, 공간 오차를 평가해야 한다. Xu 등(2024)은 GIS 데이터와 3D 프리뷰 모델을 결합한 반자동 도시 구축 파이프라인을 제시하여 수작업 모델링의 비용과 갱신 문제를 줄이는 방안을 보여준다.

UNIBUS 3D 캠퍼스는 건물, 정류장, 도로, 지형, 수면, 차량을 같은 장면에서 표현하고 운행 경로와 상태를 연결한다. 하지만 실측 BIM이나 연속 센스트림이 아니라 공개 지도와 관리자 좌표, 수작업 모델을 사용한다. 본 연구는 이를 2D/3D 디지털 트윈 기반 프로토타입으로 부르며, 운영형 디지털 트윈과 동일한 공간 정확도나 동기성을 주장하지 않는다. 이러한 구분은 학술제 시연의 시각적 효과와 연구 결과의 타당성을 동시에 지키기 위해 필요하다.

2.4 PWA와 웹 기반 3D 기술

웹 앱 매니페스트는 앱 이름, 아이콘, 시작 URL, 화면 표시 방식과 같은 설치 메타데이터를 JSON 형식으로 정의한다(World Wide Web Consortium, 2026b). 서비스 워커는 페이지 실행 여부와 독립적으로 이벤트를 처리하고 fetch와 캐시 저장소를 사용하여 오프라인 가능한 웹 애플리케이션을 구성할 수 있게 한다(World Wide Web Consortium, 2026a). 두 기술을 사용하면 앱스토

어 배포 없이 링크로 접근한 뒤 홈 화면에 설치할 수 있고, 정적 자산을 캐시하여 반복 실행 비용을 줄일 수 있다.

그러나 PWA가 모든 네트워크 기능을 오프라인으로 만드는 것은 아니다. 버스 위치와 공지, 변경된 노선은 최신 서버 상태가 중요하므로 캐시된 데이터와 실시간 데이터를 구분해야 한다. UNIBUS는 앱 셀과 정적 자산을 precache 하고 3D 대형 청크는 일반 초기 화면의 강제 로드와 service worker precache에서 분리하였다. 이 구조는 초기 접근 비용을 줄이지만 사용자가 처음 3D 화면에 진입할 때 별도의 다운로드와 파싱 비용이 발생한다.

2D 지도는 NAVER 지도 API의 Marker와 Polyline을 이용하고 (NAVER Cloud Platform, n.d.), 3D 장면은 Three.js 기반 React Three Fiber를 이용한다. 두 표현은 렌더링 엔진이 다르지만 원본 노선과 정류장 좌표, 시뮬레이션 시각을 공유한다. 좌표를 화면마다 복제하지 않고 변환 함수와 단일 데이터 혹은 통해 제공하는 것이 2D · 3D 동기화의 핵심이다.

2.5 인증과 데이터 접근 보안

브라우저에서 데이터베이스 API를 직접 사용할 때는 프론트엔드의 화면 숨김만으로 쓰기 권한을 보호할 수 없다. PostgreSQL의 행 수준 보안은 각 조회 · 삽입 · 수정 · 삭제 요청에 정책 조건을 적용하며, 정책이 없으면 기본 거부를 적용할 수 있다 (PostgreSQL Global Development Group, 2026). Supabase는 브라우저에 노출되는 public 스키마의 테이블에 RLS를 활성화하고 역할별 최소 권한을 부여하도록 권고한다 (Supabase, n.d.). UNIBUS는 사용자 화면에 필요한 노선 · 정류장 · 최신 위치는 공개 조회로 두되, 사용자 · 인증 토큰 · 운행 세션과 관리 쓰기는 Edge Function과 RLS로 제한한다.

세션 보안에는 별도의 문제가 있다. OWASP는 자바스크립트에서 항상 접근할 수 있는 localStorage에 세션 식별자를 저장하지 말고 HttpOnly 쿠키를 고려하도록 권고한다 (OWASP Foundation, n.d.). NIST SP 800-63B-4는 실패 인증 시도에 rate limiting을 적용하고 세션 비밀의 생성, 수명, 무효화를 관리하도록 요구한다 (National Institute of Standards and Technology, 2025). 현재 시

시스템은 32바이트 난수 토큰을 발급하고 서버에는 SHA-256 해시만 저장하며 서버에서 만료와 현재 역할을 확인한다. 반면 브라우저에는 30일 bearer token 을 localStorage에 보관하고 로그인 시도 제한이 없어 후속 개선이 필요하다.

2.6 선행연구의 공백과 본 연구의 위치

관련 문헌은 GPS 기반 도착 예측, 대학 셔틀 모바일 앱, 스마트 캠퍼스 3D 플랫폼을 각각 다루지만, 캠퍼스 순환·철도 연계·통학 노선의 서로 다른 운행 규칙을 하나의 모델로 통합하고 관리자 편집 결과를 2D와 3D에 동시에 연결한 사례는 제한적이다. 또한 실제 기사의 운전 중 조작을 배제한 채 차량 단말 자동 송신과 관리자 사전 설정을 중심으로 역할 흐름을 설계한 논의도 충분하지 않다.

본 연구는 새로운 ETA 알고리즘의 정확도 경쟁보다 통합 정보시스템 인공물의 구조와 구현, 검증에 초점을 둔다. 문헌의 고급 예측 기법은 현재 성능을 과장하기 위한 근거가 아니라 후속 운영 단계의 요구사항을 정의하는 기준으로 사용한다. 3D 역시 정밀 복제의 완성을 주장하기보다 동일한 노선과 실시간 상태를 공간 맥락 속에서 표현하는 인터페이스로 평가한다.

제3장 연구 방법 및 요구사항

3.1 설계과학 연구 절차

본 연구는 문제를 설명하는 데 그치지 않고 실제 작동하는 정보시스템 인공물을 만들고 평가하는 설계과학 연구로 수행하였다. Hevner 등(2004)은 설계 인공물의 관련성, 엄밀성, 평가를 함께 강조하며, Peffers 등(2007)은 문제 인식, 해결 목표, 설계·개발, 시연, 평가, 의사소통의 DSRM 절차를 제시하였다. UNIBUS 개발은 이 절차를 다음과 같이 적용하였다.

첫째, 2025년 9월 9일부터 팀이 대학 교통 서비스의 정보 단절과 발표 주제를 탐색하였다. 둘째, 2026년 3월 2일부터 본격 개발을 시작하여 사용자·관리자·기사 역할, 학내순환과 통학버스, Supabase 기반 데이터 구조를 구현하였다. 셋째, 사용자 피드백과 화면 검토를 반복하면서 신창역 셔틀 규칙, 관리자 경로 편집, 2D·3D 동기화, PWA 알림과 보안 정책을 확장하였다. 넷째, 2026년 7월 15일 기준 저장소와 연결된 원격 데이터베이스를 대상으로 정량 평가를 수행하였다.

표 3-1. 설계과학 단계와 UNIBUS 연구 활동

단계	주요 활동	산출물
문제 인식	교통 정보 분산, 경로 불일치, 기사 조작 제약 분석	문제 정의, 이해관계자 요구
목표 정의	통합 노선 모델, 역할 분리, 2D/3D 동기화	연구질문, 기능·비기능 기준
설계·개발	React PWA, Edge Function, PostgreSQL, 2D/3D 구현	실행 가능한 프로토타입
시연	학내순환 3대와 신창역 셔틀 경로 시뮬레이션	학술제용 운행 장면, 역할별 화면
평가	정적 검사, 21개 로직 시험, DB/API, Lighthouse	정량 결과, 보안 위험, 한계
의사소통	논문, 화면 캡처, 아키텍처 도식	본 논문과 재현 자료

3.2 이해관계자와 운영 제약

사용자는 현재 이용 가능한 노선과 다음 출발, 정류장, 공지를 빠르게 확인해야 한다. 관리자는 정류장 좌표와 경로, 버스와 기사 배정, 공지와 알림을 수정해야 한다. 기사는 본인에게 배정된 차량과 노선을 확인하고 운행 시작·종료 상태를 관리해야 한다. 그러나 학교 측 문의 결과 운전 중 기사가 스마트폰 앱을 직접 조작하는 방식은 현실적으로 적용하기 어렵다. 따라서 기사 화면의 존재가 운전 중 지속 입력을 전제해서는 안 된다.

이 제약에 따라 위치 갱신은 차량에 고정된 단말 또는 별도 관제 장치가 자동으로 수행하는 것을 목표 구조로 삼았다. 관리자는 운행 전에 차량과 기사를 배정하고, 기사는 정차 상태에서 배정 확인과 시작·종료만 수행한다. 운행 중 경로 진행과 최신 위치 저장은 서버와 단말이 자동 처리한다. 실제 차량 단말은 본 연구에서 설치하지 않았으므로 이 흐름은 구현 가능한 목표 구조와 시뮬레이션으로 평가한다.

표 3-2. 이해관계자별 주요 요구사항

역할	읽기 요구	쓰기 요구	안전·권한 제약
사용자	노선, 정류장, 다음 출발, 버스 상태, 공지	프로필·알림 설정	관리 데이터 수정 불가
관리자	전체 노선·버스·사용자·운행 상태	경로·정류장·배차·공지·알림 관리	모든 쓰기는 서버 역할 검사 필요
기사	배정 버스, 현재 노선, 운행 단계	운행 시작·종료, 위치 송신	배정 차량만, 운전 중 수동 조작 최소화
차량 단말	배정 노선과 활성화 운행	자동 위치·속도·방향 송신	서비스 키는 단말/서버 경계에서 보호

3.3 기능 요구사항

기능 요구사항은 통합 노선, 사용자 표시, 관리자 편집, 기사 운행, 공지·알림으로 구분하였다. 통합 노선 모델은 노선 유형, 세부 변형, 정류장 순서, 보정점, 색상, 활성화 상태, 시간표 기준, 간격, 오프셋, 연계 노선을 저장해야 한다. 사용자 화면은 학내순환과 신창역 셔틀을 하나의 셔틀 탭 안에서 전환하고,

2D와 3D가 동일한 정류장 · 차량 상태를 표시해야 한다. 통학버스는 등교 · 하교와 지역별 노선을 목록으로 제공한다.

관리자 화면은 지도에서 정류장 마커와 경로 보정점을 직접 이동하고 저장할 수 있어야 하며, 동일 이름의 정류장이 한 순환 노선에 여러 번 등장할 수 있도록 내부 식별자와 순서를 이름과 분리해야 한다. 버스 생성과 이름 수정, 기사 배정, 운행 여부 관리도 포함한다. 기사 화면은 배정된 버스가 없을 때 명확한 빈 상태를 표시하고, 공용 배차가 있다면 대상 기사에게 일관되게 노출해야 한다.

학술제 시연을 위해 여러 버스가 경로 위를 부드럽게 이동하고 2D와 3D에 같은 위치가 나타나야 한다. 학내순환은 서로 다른 위상으로 시작하는 3대를 생성하며, 신창역 셔틀은 후문 · 역 간 왕복과 대기 상태를 표현한다. 정류장에는 예정 출발, 차량에는 다음 정류장과 ETA를 표시하되 정보 라벨이 차량 자체를 가리지 않도록 화면 거리에 따른 배치가 필요하다.

3.4 비기능 요구사항

성능 측면에서는 일반 화면에서 3D 엔진을 강제로 불러오지 않고 화면 단위 지연 로딩을 적용해야 한다. 3D 장면은 모바일에서 프레임 저하를 줄이기 위해 그림자 · 후처리 · 객체 수를 제한하고, 사용자가 2D로 전환하면 렌더링 부담을 해제해야 한다. PWA는 설치 가능해야 하며 핵심 정적 자산을 캐시하되 실시간 위치를 오래된 캐시처럼 오인하지 않도록 상태 시각화를 제공해야 한다.

보안 측면에서는 관리자와 기사 API가 요청마다 인증과 현재 역할을 검사해야 하며, anon 키로 사용자 · 토큰 · 운행 세션을 직접 조회하거나 관리 데이터를 쓸 수 없어야 한다. 허용되지 않은 Origin에는 CORS 응답을 제공하지 않고, 비밀값은 Git 추적 파일에 포함하지 않아야 한다. 알려진 의존성 취약점, 고정 기본 계정, 세션 저장, 로그인 시도 제한, 브라우저 보안 헤더를 점검 대상으로 정하였다.

유지보수 측면에서는 정류장과 경로 좌표의 단일 출처, 데이터베이스 마이그레이션 이력, 공통 TypeScript 타입, 화면 단위 모듈을 사용한다. 관리자의 경

로 저장은 정류장과 보정점이 절반만 갱신되지 않도록 하나의 트랜잭션으로 수행해야 한다. 최신 위치와 장기 이력은 저장 목적과 쓰기 빈도가 다르므로 테이블을 분리해야 한다.

3.5 평가 설계

현장 사용자 시험을 수행하지 못했기 때문에 평가 질문을 프로토타입 내부 타당성에 맞추었다. 첫째, TypeScript · ESLint · production build가 통과하는가? 둘째, 시간표 · 경로 · ETA · 시뮬레이션의 경계조건이 결정적으로 재현되는가? 셋째, 원격 DB 스키마와 마이그레이션이 일치하고 비인증 요청이 역할 경계를 넘지 못하는가? 넷째, 공개 온보딩과 3D 화면의 초기 성능과 번들 비용은 어느 수준인가? 다섯째, 남은 보안 위험을 명확히 식별할 수 있는가?

알고리즘 시험은 실제 애플리케이션의 TypeScript 유틸리티를 직접 import 하여 실행하였다. 고정 시각과 합성 경로를 사용해 시간표 문자열 파싱, 10분 간격 계산, 열차 기준 오프셋, 다음 날 전환, 경로 거리 · 재표본화 · 방위각, 45초 stale 판정, 대기 ETA, 24시간 좌표 경계 등을 검증하였다. 브라우저 성능은 production preview에서 Lighthouse 기본 모바일 시뮬레이션을 사용해 공개 경로 두 개를 각각 5회 측정하고 중앙값과 범위를 보고하였다.

보안 평가는 소스 검토, npm audit, Supabase DB lint, 비인증 Edge Function 요청, anon key 직접 PostgREST 요청, CORS preflight를 결합하였다. 자동 도구에서 오류가 0이라고 해서 취약점이 없다고 결론 내리지 않고 세션 저장, rate limiting, 헤더, 로그, Edge Function 타입 검사 범위를 별도 잔여 위험으로 기록하였다.

제4장 시스템 설계

4.1 전체 아키텍처

UNIBUS는 역할별 클라이언트, React PWA 애플리케이션 계층, Supabase Edge Function 서비스 계층, PostgreSQL 데이터 계층으로 구성된다. 사용자와 기사 화면은 모바일 사용을 우선하고, 관리자 화면은 데스크톱의 넓은 작업 공간과 모바일의 반응형 접근을 함께 고려한다. 모든 화면은 하나의 React 애플리케이션에서 라우트 단위로 지연 로딩되며 인증 상태와 역할에 따라 보호된 Outlet을 통과한다.

UNIBUS 시스템 아키텍처

역할별 클라이언트, 애플리케이션 서비스, 데이터 계층의 연결

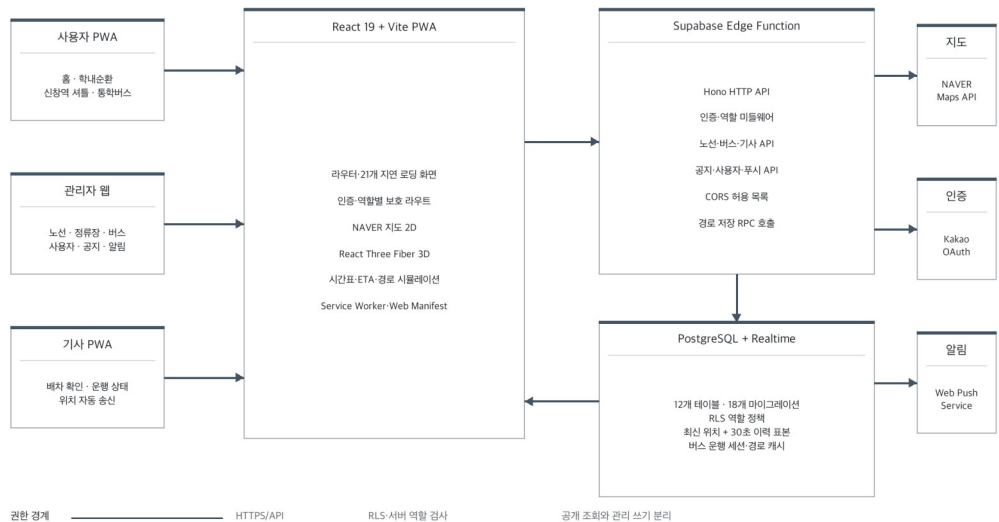


그림 4-1. UNIBUS 시스템 아키텍처

사용자 PWA는 홈, 셔틀, 통학버스, 공지, 설정을 제공한다. 셔틀 화면은 학내순환과 신창역 셔틀을 상단 세그먼트로 전환하고 NAVER 지도 2D와 React Three Fiber 3D를 같은 화면 안에서 선택한다. 관리자 웹은 노선 · 정류장, 버스, 사용자, 공지, 알림, 3D 관리와 시뮬레이션을 제공한다. 기사 PWA는 배정 버스와 현재 운행을 표시한다.

Edge Function은 Hono 기반 HTTP API로 인증, 버스, 노선, 기사, 사용자, 공지, 캠퍼스, 알림 엔드포인트를 제공한다. 인증·역할 미들웨어는 bearer token의 해시, 만료, 사용자 현재 역할을 요청마다 확인한다. CORS는 설정된 Origin과 UNIBUS Vercel 배포 패턴, localhost 개발 Origin만 허용한다. 관리 쓰기 요청은 Edge Function을 통과하고, 공개 노선과 최신 위치 같은 제한된 자료만 RLS 정책을 통해 anon 조회가 가능하다.

PostgreSQL은 관계형 기준정보와 운행 상태를 저장하며 Supabase Realtime은 최신 버스 상태 변경을 클라이언트에 전달한다. 외부 서비스는 NAVER 지도, Kakao OAuth, Web Push로 분리된다. 외부 API 오류가 핵심 데이터 모델을 변경하지 않도록 지도 렌더링과 노선 저장, 인증 제공자와 내부 사용자 역할을 구분하였다.

4.2 데이터 모델

데이터베이스는 12개 핵심 테이블로 구성된다. users는 사용자 식별자와 역할을 저장하고 auth_tokens, push_subscriptions, notices와 연결된다. routes는 노선 기준정보와 운행 규칙을 저장하며 route_stops, route_shape_points, route_path_cache와 연결된다. buses는 현재 노선과 배정 기사를 참조하고 bus_trips, bus_latest_state, bus_locations를 통해 운행 세션과 위치를 관리한다.

UNIBUS 핵심 데이터 모델

12개 관계형 테이블과 실시간 위치 저장 구조

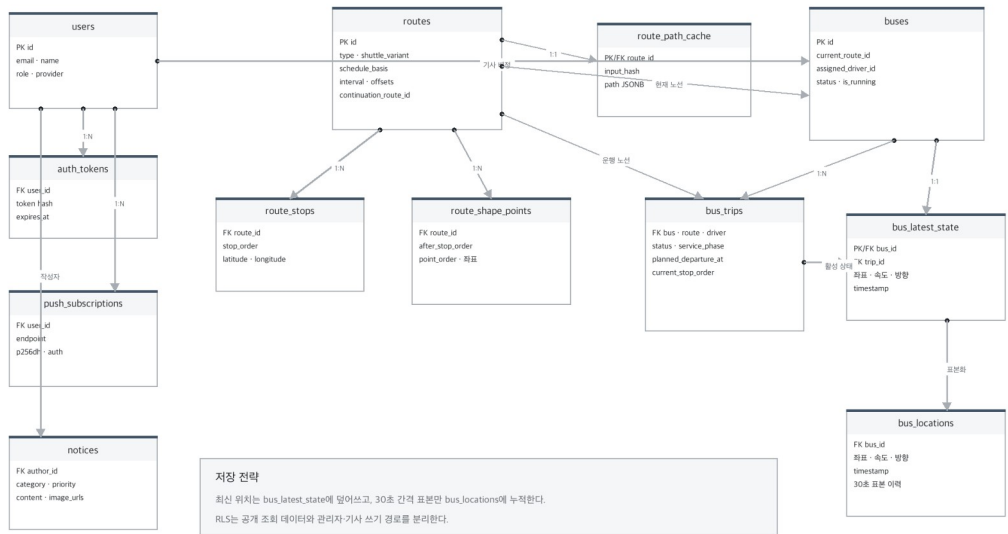


그림 4-2. UNIBUS 핵심 데이터 모델

정류장 이름은 사용자에게 보이는 표시값일 뿐 고유키로 사용하지 않는다. 한 순환 노선이 출발지로 돌아오면 동일한 후문 정류장이 시작과 종료에 다시 나타날 수 있기 때문이다. 각 정류장은 UUID와 stop_order로 구분하고, 경로 보정점은 after_stop_order와 point_order로 정류장 사이의 도로 형태를 표현한다. 이 설계로 후문1, 후문2처럼 사용자에게 어색한 임시 이름을 만들지 않고도 동일 장소의 반복 방문을 저장할 수 있다.

최신 위치와 위치 이력은 분리한다. bus_latest_state는 버스별 한 행을 유지하며 새 위치가 들어올 때 덮어써서 Realtime 구독과 현재 화면 조회 비용을 줄인다. bus_locations는 분석과 재생에 필요한 이력을 저장하되 30초보다 자주 누적하지 않는다. record_bus_location 함수는 최신 상태를 항상 갱신하고 마지막 이력 시각이 30초 이상 지난 경우에만 이력 행을 추가한다. 일부 호출에서 30일보다 오래된 이력을 확률적으로 정리하여 별도 스케줄러가 없는 프로토타입에서도 누적량을 제한한다.

bus_trips는 앱 재시작 이후에도 현재 운행 단계와 정류장 진행을 복구하기 위한 세션 테이블이다. 하나의 버스와 기사에 활성 운행이 하나만 존재하도록

부분 유니크 인덱스를 두었다. `service_phase`는 운행 중, 역 대기, 역 방향 이동, 학교 방향 이동, 학내순환, 주차장 복귀를 구분한다. 이 값은 신창역 셔틀이 후문 도착 후 종료하는지 학내순환을 한 번 연결하는지 표현하는 데 사용한다.

4.3 노선과 운행 규칙 모델

노선은 `type`과 `shuttle_variant`를 함께 사용한다. `type`은 캠퍼스 셔틀과 통학버스 같은 상위 분류를 나타내고, `shuttle_variant`는 학내순환, 학교에서 역, 역에서 학교 종료, 역에서 학교 도착 후 학내순환 연계를 구분한다. 시간표의 의미는 `schedule_basis`로 분리하여 버스 출발 시각, 열차 출발 시각, 열차 도착 시각 중 하나를 선택한다.

표 4-1. 셔틀 운행 규칙의 데이터 표현

운행 유형	기준	간격(분)	출발 오 프셋	승차 대 기	후속 운행
학내순환	<code>bus_departure</code>	10	0	0	없음
후문 → 신창역	<code>train_departure</code>	없음	10분 전	0	없음
신창역 → 후문 종료	<code>train_arrival</code>	없음	0	5분 후	없음
신창역 → 후문 → 학내순환	<code>train_arrival</code>	없음	0	5분 후	학내순환 1회

시간표 문자열을 파싱할 때 유효한 HH:mm 값만 선택하고 분 단위로 정렬한다. 학내순환은 하루의 기준 시작시각에서 10분 간격으로 다음 출발을 계산한다. 후문발은 열차 출발 시각에서 10분을 빼고, 역발은 열차 도착 시각에 5분을 더한다. 당일 마지막 운행이 지났다면 다음 날 첫 시각을 선택한다. 날짜 경계를 명시적으로 처리하지 않으면 자정 근처에서 이미 지난 시각을 다음 출발로 표시할 수 있으므로 알고리즘 시험에 이 조건을 포함하였다.

연계 운행은 원래 신창역 노선과 현재 실행 노선을 구분한다. `origin_route_id`는 사용자가 선택한 셔틀 서비스를 유지하고, `route_id`와 `service_phase`는 차량이 현재 후문 복귀 또는 학내순환 구간에 있는지 나타낸다.

이 구조를 사용하면 하나의 차량이 서비스 중 노선을 전환하더라도 사용자에게는 한 번의 연속 운행으로 표시할 수 있다.

4.4 경로 편집과 동기화

관리자가 지도에서 정류장과 경로 보정점을 이동하면 프론트엔드는 좌표, 순서, 표시 이름을 구조화된 배열로 전송한다. Edge Function은 `replace_route_details` RPC를 호출하여 기존 정류장과 보정점을 삭제하고 새 배열을 하나의 트랜잭션 안에서 삽입한다. 이 방식은 정류장 저장 후 보정점 저장이 실패해 경로가 절반만 바뀌는 상태를 방지한다.

노선 편집과 실시간 운행 정보 흐름

관리자 설정부터 사용자 2D/3D 표시까지 동일한 데이터 계약을 사용



그림 4-3. 노선 편집과 실시간 운행 정보 흐름

사용자 2D와 3D 화면은 같은 노선 조회 결과를 사용한다. 정류장 사이에는 순서에 맞는 보정점을 삽입하여 최종 `polyline`을 만들고, 너무 가까운 중복점은 제거한 뒤 일정 간격으로 재표본화한다. 시뮬레이션 버스는 전체 경로의 누적 거리에서 시간에 따른 진행 거리를 계산하여 두 표본점 사이를 선형 보간한다. 방위각은 다음 표본점 방향으로 계산한다.

2D 지도는 위도·경도를 지도 API에 직접 전달하고 마커의 픽셀 기준점만 아이콘 `anchor`로 사용한다. 좌표와 픽셀 `offset`을 섞지 않기 때문에 확대·축소

시 정류장이 이동해 보이는 현상을 방지한다. 3D에서는 기준 위도 · 경도와 미터 환산 상수를 사용해 로컬 XZ 좌표로 변환하고 지형 높이에 작은 offset을 더한다. 경로 선과 지형 표면이 같은 깊이에 놓이면 z-fighting이 발생하므로 경로를 표면 위에 분리하되 실제 도로와 겹쳐 보이는 과도한 높이는 피한다.

4.5 2D · 3D 표현 설계

2D 지도는 전체 경로 파악과 정확한 위치 확인에 적합하고, 3D는 건물 · 고저차 · 정류장과 차량의 공간 관계를 이해하는 데 적합하다. 두 모드는 경쟁 관계가 아니라 같은 상태의 다른 표현으로 설계하였다. 사용자가 모드를 전환할 때 현재 선택한 노선과 버스, 정류장을 유지하고 카메라 전환 애니메이션을 적용하여 맥락 단절을 줄인다.

3D 카메라의 기본 시점은 서비스별 이용 맥락에 따라 다르다. 학내순환은 후문을 전경으로 두면서 향설생활관 3과 정문 방향까지 볼 수 있는 거리에서 시작한다. 신창역 셔틀은 후문과 김승우 라운지 주변을 보여주되 지나치게 가까이 확대하지 않고 캠퍼스와 역 방향을 함께 이해할 수 있는 시점을 사용한다. 정류장을 클릭하면 해당 위치를 향해 이동하고, 버스를 클릭하면 차량을 화면 중앙에 두되 라벨과 차량이 겹치지 않는 거리에서 추적한다.

정류장 라벨은 네이비 배경과 흰색 텍스트로 대비를 높이고, 버스 라벨은 같은 크기 계열을 사용하여 모바일에서도 읽히도록 한다. 차량 모델은 정류장 마커보다 약간 크게 하되 건물과 도로를 가리지 않도록 제한한다. 건물과 지형의 명도가 비슷하면 형태가 구분되지 않으므로 지형은 낮은 채도의 연녹색, 도로는 중성 회색, 건물은 밝은 흰색 · 회색과 제한된 포인트색을 사용한다. 3D 화면은 앱의 다크모드와 무관하게 지도 · 공간 해석에 적합한 고정 밝은 팔레트를 유지한다.

4.6 인증 · 권한 및 보안 설계

로그인 시 서버는 bcrypt 해시를 검증하고 32바이트 CSPRNG 토큰을 생성한다. 토큰 원문은 한 번만 클라이언트에 전달하고 데이터베이스에는 SHA-256

해시를 저장한다. 이후 요청은 헤더의 토큰을 같은 방식으로 해시하여 `auth_tokens`와 `users`를 조회하고 만료와 현재 역할을 확인한다. 사용자의 역할이 관리자로 바뀌거나 취소되면 기존 토큰에 내장된 오래된 역할을 신뢰하지 않고 DB의 현재 값을 사용한다.

관리자 API는 `requireAdmin`, 기사 API는 `requireDriver` 또는 배정 검사를 통과해야 한다. 기사 위치 갱신은 활성 운행과 배정 차량을 확인한 뒤 서비스 역할로 위치 RPC를 호출한다. `record_bus_location`의 PUBLIC 실행 권한은 회수하고 `service_role`만 실행할 수 있게 하여 anon 사용자가 임의의 위치를 직접 기록하지 못하도록 한다.

RLS는 방어 심층화를 제공한다. 공개 사용자 화면에 필요한 노선, 정류장, 경로 보정점, 버스, 최신 위치에는 SELECT 정책을 두고, 사용자 개인정보, 토큰, 푸시 구독, 운행 세션에는 anon 직접 조회 정책을 두지 않는다. 쓰기 정책은 프론트엔드에서 제거하고 Edge Function이 서비스 역할로 수행한다. CORS와 Vercel의 X-Content-Type-Options, X-Frame-Options도 적용하였다.

검토 과정에서 초기 시드 코드와 과거 SQL에 예측 가능한 기본 자격증명이 남아 있음을 발견하였다. 현재 트리에서는 고정 관리자와 샘플 사용자를 제거하고, SEED_ADMIN_EMAIL과 15자 이상 SEED_ADMIN_PASSWORD를 Edge Function secret으로 제공한 경우에만 관리자 시드를 실행하도록 수정하였다. 응답에는 자격증을 반환하지 않는다. 다만 Git 이력에 존재했던 값은 공개된 것으로 간주해야 하므로 원격 관리자 비밀번호 회전과 기존 세션 무효화가 별도로 필요하다.

제5장 구현

5.1 개발 환경과 기술 스택

프론트엔드는 React 19.1, TypeScript 5.8, Vite 7.3을 사용하였다. React Router 6은 사용자 · 기사 · 관리자 라우트를 분리하고, 모든 주요 화면은 lazy와 Suspense로 지연 로딩한다. Tailwind CSS 4와 Radix UI 계열 컴포넌트, Lucide 아이콘을 사용하여 모바일과 데스크톱 UI를 구성하였다. 애니메이션은 Framer Motion 11을 사용하였다.

2D 지도는 NAVER 지도 API v3, 3D 장면은 Three.js 0.185, React Three Fiber 9.6, Drei 10.7을 사용하였다. 서버는 Supabase Edge Function의 Deno 런타임과 Hono를 사용하고, 데이터는 Supabase PostgreSQL과 Realtime에 저장한다. PWA는 Vite Plugin PWA를 사용해 매니페스트와 Workbox 기반 서비스 워커를 생성한다.

표 5-1. 주요 기술 스택과 역할

계층	기술	버전	역할
UI	React, TypeScript, Tailwind CSS	19.1, 5.8, 4.1	역할별 화면과 상태 표현
빌드 · PWA	Vite, Vite Plugin PWA	7.3, 1.3	번들링, 지연 로딩, 설치 · 캐시
2D 지도	NAVER Maps JavaScript API	v3	지도, 마커, polyline, 경 로 편집
3D	Three.js, React Three Fiber, Drei	0.185, 9.6, 10.7	캠퍼스 · 차량 · 카메라 렌 더링
API	Supabase Edge Function, Hono	Deno, npm	인증, 역할, 업무 API
데이터	PostgreSQL, Supabase Realtime	Supabase	RLS, 노선 · 운행 · 최신 위치

5.2 사용자 화면

온보딩은 여러 장을 넘기는 초기 구조에서 하나의 긴 스크롤 페이지로 변경하였다. 사용자가 처음 보는 화면에서 UNIBUS 이름, 셔틀 위치와 시간표 확인이라는 핵심 기능, 실제 앱 화면 미리보기, 시작 버튼을 순서대로 보여준다. 스크롤에 따라 구성요소가 자연스럽게 나타나도록 하되 QR처럼 구현되지 않은 기능이나 관리자·기사 설명은 제거하였다.



사용자 중심 단일 스크롤 온보딩 화면

그림 5-1. 사용자 중심 단일 스크롤 온보딩 화면

홈은 다음 운행 요약, 신창역 전철 시간표 진입, 공지사항과 빠른 이동을 제공한다. 사용하지 않는 즐겨찾기와 실시간 추적 카드는 제거하여 첫 화면의 정보 밀도를 조정하였다. 전철 시간표는 평일·주말·공휴일 탭과 방향을 구분할 수 있는 구조로 만들었으며, 실제 운영 적용 전 공식 시간표 갱신 절차가 필요하다.

셔틀 화면은 하단 내비게이션의 학내순환을 셔틀버스로 확장하고 상단에서 학내순환과 신창역 셔틀을 전환한다. 학내순환 화면은 2D에서 전체 노선을 먼저 보여주고 3D에서는 후문에서 캠퍼스 전체를 바라보는 시점으로 시작한다. 신창역 셔틀 화면은 김승우 라운지 정류장, 다음 출발까지 남은 시간, 연결 열차 시각, 역 방향 노선을 중심으로 구성한다. 통학버스는 실시간 운행 카드를 반복해 보여주지 않고 노선 목록과 등하교 시간표에 집중한다.



그림 5-2. UNIBUS 사용자 모바일 주요 화면

색상은 전체 앱에서 흰색과 네이비를 기본으로 하고, 다크모드는 iOS 계열의 검정 · 짙은 회색 배경과 눈부심이 적은 연한 파란색 강조색을 사용한다. 시스템 자동 모드는 제외하고 사용자가 화이트 · 다크를 직접 선택하도록 하였다. 3D 장면은 색상 판독과 지도 일관성을 위해 다크모드를 적용하지 않는다.

5.3 2D 지도와 마커 구현

NAVER 지도 스크립트는 필요한 시점에 한 번만 로드하고, 지도 인스턴스와 마커 · polyline 객체는 React 생명주기에 맞춰 생성 · 정리한다. 사용자 화면에서 전체 보기 시 fitBounds를 사용하되 지도 컨테이너가 실제 크기를 얻은 뒤 bounds를 적용한다. 모바일 하단 시트와 상단 컨트롤이 지도를 덮는 영역을 고려해 padding을 추가한다. 초기 학내순환 2D는 전체 노선보다 한 단계 확대된 시점을 원한다는 요구에 따라 bounds 계산 후 제한된 zoom 보정을 적용하였다.

정류장 마커는 학내순환과 통학버스에서 같은 아이콘 컴포넌트와 anchor 규칙을 사용한다. 확대 · 축소 시 위치가 달라 보였던 원인은 HTML 마커의 시각적 중심과 지도 좌표 anchor가 일치하지 않았기 때문이다. 마커 크기와 anchor를 고정 픽셀로 설정하고 위치는 위도 · 경도만 갱신하도록 분리하였다. 버스 마

커는 진행 방향을 회전으로 표현하고 일반 대중교통 앱에서 익숙한 버스 전면 아이콘을 사용한다.

관리자 경로 편집기에서는 정류장 마커와 경로 보정점을 드래그할 수 있다. 보정점은 승객에게 정류장으로 보이지 않고 도로 모양을 만드는 용도로만 사용한다. 관리자가 학내순환과 신창역 셔틀을 선택하면 각각의 노선 ID를 기준으로 별도 좌표를 불러오므로 한 노선의 수정이 다른 노선에 섞이지 않는다. 저장 후에는 쿼리 캐시와 3D 경로 데이터를 무효화하여 사용자 화면이 같은 값을 다시 읽는다.

5.4 3D 캠퍼스와 차량 시뮬레이션

3D 프로토타입은 캠퍼스 지형, 주요 건물, 도로, 정문 · 후문 구조물, 김승우 라운지 주변, 신창역과 연결 도로, 인접 수면을 단순화된 메시로 구성한다. 지형은 고도 차이를 완전히 평평하게 만들지 않고 완만한 높이 변화를 적용하며, 가장자리가 절벽처럼 끊겨 공중에 떠 보이지 않도록 캠퍼스 밖까지 넓은 기반 지형을 확장한다. 수면은 사실적 셰이더 대신 낮은 채도의 연한 하늘색 면으로 표현하여 렌더링 비용과 시각적 혼란을 줄였다.

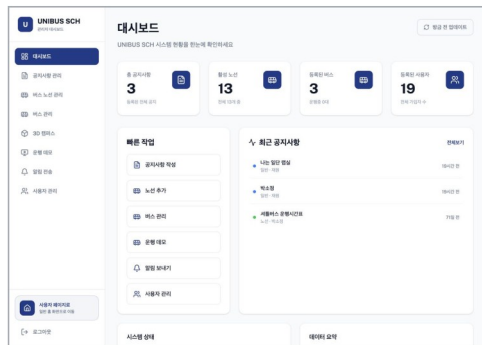
건물과 구조물은 실제 위치와 형태를 최대한 참고하되 측량 · BIM 자료가 없는 부분은 발표용 시각화 수준으로 단순화하였다. 후문 구조물과 정문, 신창역은 사진 한 장의 원근을 그대로 복제하지 않고 공개 지도와 여러 시점을 비교해 위치 관계를 조정하였다. 정확성을 검증할 수 없는 세부 구조는 과도하게 묘사하지 않는다.

경로는 `route polyline`을 일정 거리로 재표본화한 뒤 시간에 따른 진행률을 누적 거리로 변환한다. 버스는 인접 표본점 사이를 보간하고 방위각에 따라 회전한다. 학내순환 3대는 동일한 경로에서 다른 위상으로 시작하여 동시에 움직인다. 브라우저 탭이 잠시 비활성화된 뒤 큰 `delta`가 들어와 차량이 순간 이동하지 않도록 시간 차를 제한하고, 화면의 `requestAnimationFrame`에 맞춰 위치를 갱신한다.

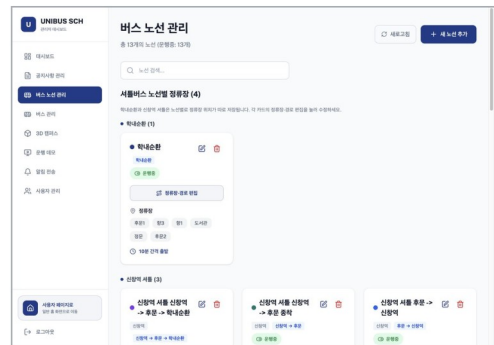
2D와 3D는 같은 시뮬레이션 시각과 경로 샘플을 사용한다. 따라서 사용자가 모드를 전환해도 차량이 다른 정류장으로 뛰지 않는다. 3D에서 버스나 정류장을 클릭하면 선택 ID를 상위 상태로 전달하고 카메라 목표점을 보간한다. OrbitControls의 회전·이동 방향은 모바일 드래그 감각에 맞게 설정하고, 너무 가까운 최소거리와 지나치게 넓은 최대거리를 제한한다.

5.5 관리자와 기사 화면

관리자 대시보드는 버스, 노선, 사용자, 공지 수와 최근 공지를 한 화면에서 보여준다. 노선 관리에서는 서비스 유형별 노선 카드, 정류장 순서, 지도 편집, 시간표 규칙을 함께 관리한다. 버스 관리에서는 버스 ID, 이름, 유형, 정원, 번호판, 상태, 현재 노선, 배정 기사를 생성·수정한다. 사용자 관리에서는 이름과 역할을 변경할 수 있고, 역할 칩은 좁은 화면에서도 글자가 세로로 꺾이지 않도록 최소 너비와 nowrap을 적용하였다.



(a) 관리자 대시보드



(b) 노선-정류장 관리

동일한 노선-정류장 데이터가 사용자 2D/3D 화면과 관리자 편집기에 연결된다.

그림 5-3. 관리자 대시보드와 노선 관리 화면

관리자 3D 탭은 학내순환과 신창역 셔틀의 정류장 위치를 3D에서 확인하고 이동할 수 있는 진입점을 제공한다. 정밀 경로 편집은 2D 지도에서 수행하고, 3D에서는 건물과 정류장의 상대 위치를 검토하는 역할로 구분하였다. 데스크

크톱에서는 고정 사이드바와 넓은 작업 영역을 사용하고, 모바일에서는 사이드바를 접을 수 있는 내비게이션과 단일 열 폼으로 전환한다.

기사 홈은 배정 버스가 있는 경우 차량과 노선, 상태를 보여주고 운행 시작으로 이동한다. 배정이 없을 때는 오류처럼 보이는 빈 화면 대신 배정된 버스가 없습니까와 관리자가 배정해야 한다는 상태를 표시한다. 운행 화면은 현재 노선, 다음 정류장, 위치 송신 상태, 시작 · 종료 명령을 제공한다. 실제 운영에서는 위치 송신을 차량 단말이 자동 수행하고 기사 화면은 정차 중 확인 용도로 제한한다.

5.6 공지, 알림과 PWA

공지는 일반, 노선, 시스템, 분실물 범주와 중요도를 가지며 이미지와 본문 하단 내용을 저장할 수 있다. 관리자가 공지를 작성하면 사용자 화면에서 고정 공지와 최신 공지를 구분한다. Web Push 구독은 사용자별 endpoint, 공개키, 인증값, user agent, 활성 상태와 마지막 오류를 저장한다. 알림 전송 실패 시 구독을 무한 반복하지 않도록 오류 상태를 기록한다.

PWA 빌드는 65개 정적 항목, 약 1,300.30 KiB를 precache한다. 3D vendor와 독립 3D 페이지의 대형 청크는 일반 앱 실행에서 필요하지 않으므로 precache 대상에서 제외한다. 새 서비스 워커가 설치되었을 때 사용자의 화면을 갑자기 재시작하지 않고 업데이트 상태를 관리하며, 알림 권한은 앱 첫 실행에서 무조건 강요하지 않고 사용자가 알림 기능을 활성화하는 맥락에서 요청한다.

5.7 개발 협업과 형상관리

개발은 GitHub 기반 팀 프로젝트로 수행하였다. 권재원은 PM과 프론트엔드·백엔드 총괄, 박소정은 백엔드, 손성윤은 프론트엔드, 김동하는 데이터베이스, 정영선과 강민영은 백엔드를 담당하였다. 실제 커밋 수는 계정과 이메일 설정, 공동 작업, 병합 방식의 영향을 받으므로 개인 기여량의 정량 지표로 사용하지 않는다. 역할 설명과 주요 구현 책임은 팀이 합의한 업무 분장을 기준으로 한다.

2026년 7월 15일 기준 저장소에는 274개 커밋이 있으며 본격 개발 시작일인 2026년 3월 2일 이후 261개가 작성되었다. GitHub Pull Request는 56개이고 그중 53개가 병합되었다. 기능 변경은 별도 브랜치에서 검증하고 PR로 메인 브랜치에 병합하며, Vercel 배포 결과를 확인하는 흐름을 사용하였다. 이 수치는 프로젝트의 변경 이력을 설명할 뿐 코드 품질이나 개인 생산성을 의미하지 않는다.

제6장 평가 결과

6.1 평가 환경

평가는 2026년 7월 15일 macOS 26.5, Apple M4, 메모리 16 GiB 환경에서 수행하였다. Node.js 24.13.1, npm 11.8.0, Lighthouse 13.4.0, Supabase CLI 2.75.0을 사용하였다. 브라우저 성능은 로컬 production build를 vite preview로 실행하고 Lighthouse 기본 모바일 시뮬레이션에서 측정하였다. 데이터베이스 검사는 프로젝트에 연결된 원격 Supabase를 대상으로 하였다.

표 6-1. 평가 환경과 도구

구분	환경 · 버전	평가 용도
하드웨어	Apple M4, 16 GiB	빌드 · 로컬 브라우저 측정
운영체제	macOS 26.5 Build 25F71	실행 환경
런타임	Node 24.13.1, npm 11.8.0	정적 검사, 시험, 빌드
브라우저 도구	Lighthouse 13.4.0	모바일 초기 로딩 반복 측정
DB 도구	Supabase CLI 2.75.0	migration list, linked DB lint
기준일	2026-07-15 KST	결과의 시간 범위

6.2 저장소 규모와 정적 품질

프론트엔드 TypeScript · TSX는 주석과 빈 줄을 포함한 물리적 줄 수 기준 24,122 LOC, Edge Function은 3,550 LOC, 마이그레이션 SQL은 1,180 LOC, CSS는 365 LOC였다. 애플리케이션 라우트는 24개, 지연 로딩 화면은 21개, Edge Function HTTP 엔드포인트는 39개, DB 테이블은 12개, 마이그레이션은 18개였다. LOC는 시스템 규모를 설명하기 위한 값이며 품질이나 개인 기여량으로 해석하지 않는다.

표 6-2. 저장소 규모와 검사 결과

항목	결과	해석 범위
프론트엔드 TS/TSX	24,122 LOC	주식 · 빈 줄 포함 물리 줄 수
Edge Function TS/TSX	3,550 LOC	서버 API 코드 규모
마이그레이션 SQL	1,180 LOC	18개 마이그레이션
앱 라우트 / 지연 화면	24 / 21	화면 단위 분리 정도
Edge 엔드포인트	39	HTTP 메서드 · 경로 수
TypeScript 검사	오류 0	`tsc --noEmit` 범위
ESLint	오류 0	저장소 ESLint 설정 범위
npm audit	취약점 0	알려진 패키지 DB 기준

npm run typecheck와 npm run lint는 오류 없이 통과하였다. Production build는 2,678개 모듈을 변환하고 약 2.6초에 완료되었다. npm audit --json은 782개 dependency record에서 알려진 취약점을 보고하지 않았다. 그러나 Edge Function 진입 파일에는 Deno 글로벌 타입 때문에 @ts-nocheck가 있으며 npm TypeScript 검사는 Deno 계약을 완전히 검사하지 않는다. deno check는 저장소에 npm:bcryptjs를 해석할 Deno 의존성 설정이 없어 단독 실행되지 않았고, 이는 도구 범위의 한계로 기록하였다.

6.3 알고리즘 시나리오 시험

실제 애플리케이션 유틸리티를 Node의 TypeScript strip 기능으로 불러와 21개 시나리오를 실행하였다. 시간표 · 운행 규칙 7개, 경로 이동 5개, ETA 4개, 시뮬레이션 5개가 모두 통과하였다. 시험은 고정 입력을 사용하며 실행마다 같은 결과를 반환한다.

표 6-3. 알고리즘 시나리오 시험 결과

영역	시험 수	통과	주요 경계조건
시간표 · 운행 규칙	7	7	10분 간격, 열차 전 10분 · 후 5분, 다음 날

영역	시험 수	통과	주요 경계조건
경로 거리 · 이동	5	5	중복점 제거, 재표본화, 중간점, 방위각
ETA 상태	4	4	45초 stale, 대기 출발, 운행 없음
시뮬레이션	5	5	다중 버스, 결정성, 24시간 좌표 경계
합계	21	21	실패 0

시간표 시험은 유효한 시각만 정렬하고 정확한 경계 시각 직후 다음 10분 간격으로 넘어가는지 확인하였다. 후문발은 열차 출발 10분 전, 역발은 열차 도착 5분 후를 계산하고 당일 운행이 끝나면 다음 날 첫 시각을 선택하였다. 경로 시험은 위도 0.001도의 거리 근사, 0.5 m 이하 중복점 제거, 지정 간격 재표본화, 경로 절반 지점, 정북 방위각을 확인하였다.

ETA 시험은 최신 위치의 다음 정류장 계산, 45초 초과 위치의 stale 처리, 운행 버스가 없을 때의 대기 상태, 역 대기 중 계획 출발시각 합산을 다루었다. 시뮬레이션은 학내순환 3대와 신창역 셔틀을 24시간 동안 3분 간격으로 표본화하여 정의된 경계 상자를 벗어나지 않는지 확인하였다. 이 시험은 합성 경로의 소프트웨어 로직을 검증하며 실제 GPS 오차나 교통 정체에 대한 ETA 정확도를 의미하지 않는다.

6.4 데이터베이스와 API 권한 시험

supabase migration list에서 로컬과 원격의 18개 마이그레이션 버전이 모두 일치하였다. supabase db lint --linked --level warning --fail-on error는 public과 extensions 스키마에서 오류를 보고하지 않았다. 이는 현재 연결된 원격 스키마가 정적 DB 검사 규칙을 통과했음을 뜻하지만 데이터 내용과 모든 권한 조합의 안전성을 보장하지 않는다.

비인증 요청에서는 공개 노선 조회만 200을 반환하고 사용자 관리, 기사 상태, 노선 생성은 401을 반환하였다. 등록되지 않은 Origin에서 보낸 CORS preflight에는 Access-Control-Allow-Origin이 포함되지 않았다. anon key로 PostgREST를 직접 조회했을 때 users와 auth_tokens는 200과 빈 배열을 반환했

고, 공개 routes는 1행을 반환하였다. routes INSERT는 401로 거부되었다. RLS가 행을 숨기는 SELECT는 200과 빈 배열을 반환할 수 있으므로 상태 코드와 행 수를 함께 확인하였다.

표 6-4. DB와 비인증 API 시험 결과

시험	기대	결과	판정
로컬 · 원격 migration version	일치	18 / 18	통과
linked DB lint	error 0	error 0	통과
GET /routes	200	200	공개 조회
GET /users	401	401	거부
GET /driver/status	401	401	거부
POST /routes	401	401	거부
악성 Origin preflight	ACAO 없음	없음	거부
anon routes INSERT	401	401	거부

6.5 빌드 산출물과 브라우저 성능

Production build에서 공통 CSS는 161.99 kB, gzip 26.66 kB였다. 가장 큰 청크는 campus-3d-vendor로 원본 944.52 kB, gzip 256.47 kB였으며 Vite의 500 kB 경고 기준을 넘었다. react-vendor는 gzip 81.31 kB, Campus3DScene은 59.66 kB, Supabase 클라이언트는 54.96 kB, motion-vendor는 37.78 kB였다. 3D 엔진과 장면이 일반 초기 번들과 분리된 것은 긍정적이지만 3D 첫 진입 비용은 여전히 크다.

Lighthouse는 /onboarding과 /campus-3d를 각각 5회 측정하였다. 온보딩 Performance 중앙값은 79점, 범위는 77-81점이었고 3D는 중앙값 73점, 범위 72-75점이었다. 온보딩 FCP와 LCP 중앙값은 3.678초와 3.903초, 3D는 4.128초와 4.654초였다. 두 경로의 Accessibility와 Best Practices는 모두 100점, TBT는 0 ms, CLS는 0이었다.

프로토타입 정량 평가 요약

2026-07-15, production build와 Lighthouse 기본 모바일 시뮬레이션

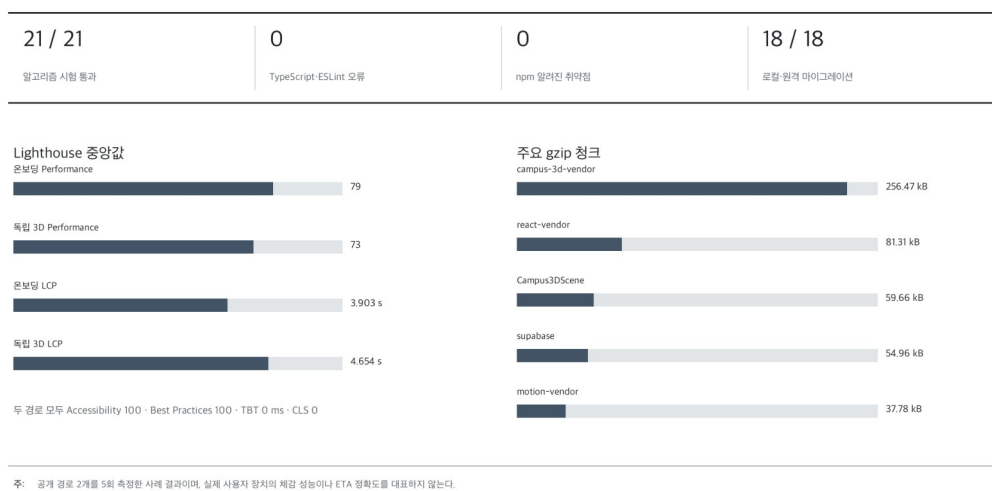


그림 6-1. 프로토타입 정량 평가 요약

표 6-5. Lighthouse 5회 반복 측정 중앙값

경로	Performance	Accessibility	Best Practices	FCP	LCP	TBT	CLS
/onboarding	79	100	100	3.678 s	3.903 s	0 ms	0
/campus-3d	73	100	100	4.128 s	4.654 s	0 ms	0

TBT와 CLS가 0인 것은 측정한 초기 로딩에서 메인 스레드 장기 차단과 레이아웃 이동이 낮았음을 보여준다. 반면 LCP는 두 화면 모두 개선 여지가 있고 3D가 온보딩보다 0.751초 길었다. 이 측정은 공개 경로 두 개와 Lighthouse 시뮬레이션의 사례 결과이며 로그인 이후 화면, 실제 저사양 휴대전화, 캠퍼스 무선망을 대표하지 않는다.

6.6 보안 검토 결과

확인된 방어는 난수 토큰과 DB 해시 저장, 서버 만료 · 현재 역할 확인, 관리자 · 기사 미들웨어, 위치 RPC의 service role 제한, RLS, CORS 허용 목록,

환경 파일 Git 제외, 기본 Vercel 헤더, 알려진 npm 취약점 0건이다. 초기 시드의 고정 자격증명은 현재 트리에서 제거하였다.

잔여 위험은 세션, 로그인 시도, 브라우저 헤더, 개인정보 로그, Edge Function 타입 검사로 구분하였다. 가장 큰 운영 조치는 Git 이력에 노출된 기존 기본 비밀번호가 원격에서 사용되지 않는지 확인하고 관리자 비밀번호를 회전하며 기존 토큰을 무효화하는 것이다. 이 검토에서는 실제 원격 관리자 비밀번호를 시험하지 않았으므로 계정이 현재 취약하다고 단정하지 않는다.

표 6-6. 잔여 보안 위험과 개선 우선순위

ID	심각도	위험	현재 상태 · 권고
H-01	높음	과거 기본 관리자 자격증명	소스 제거 완료, 원격 비밀번호 회전 · 세션 무효화 확인
M-01	중간	localStorage의 30일 bearer token	HttpOnly · Secure · SameSite 쿠키/BFF로 이전
M-02	중간	로그인 rate limiting 부재	계정 · IP 기반 제한과 경보 추가
L-01	낮음	CSP · Referrer · Permissions 정책 부족	외부 서비스 Origin을 반영해 단계 적용
L-02	낮음	인증 로그의 이메일 · 이름	축약 ID와 추적 ID 사용, 보존 기간 제한
L-03	낮음	Deno 함수 타입검사 공백	deno.json과 CI `deno check` 구성

본 보안 결과는 소스, 스키마, 알려진 패키지 취약점, 비인증 요청을 중심으로 한 제한적 검토다. 전문 침투시험, 클라우드 조직 권한, 소셜 엔지니어링, DDoS, 푸시 공급자 보안은 포함하지 않는다. 따라서 취약점이 없다고 아니라 검사 범위에서 확인된 방어와 잔여 위험으로 해석해야 한다.

6.7 연구질문별 결과 요약

RQ1에 대해서는 공통 노선 · 정류장 · 보정점 모델과 운행 규칙, 역할별 라우트 · API, 동일 데이터 기반 2D · 3D 표현을 구현함으로써 통합 구조를 제시하였다. 학내순환과 신창역 셔틀은 상위 셔틀 화면에서 전환되고, 통학버스는

같은 사용자 앱의 별도 탭으로 제공된다. 관리자 경로 저장에 사용자 화면과 같은 데이터 테이블을 갱신한다.

RQ2에 대해서는 21개 알고리즘 시험, 정적 검사, 빌드, DB/API 권한 시험이 정의한 내부 기능 기준을 충족하였다. 그러나 Lighthouse Performance는 73-79점 중앙값, LCP는 3.903-4.654초였고 3D vendor 청크가 가장 컸으므로 성능이 완성되었다고 볼 수 없다. 실제 ETA 정확도와 현장 사용성은 평가하지 못했다.

RQ3에 대해서는 관리자 사전 배정, 차량 단말 자동 위치 송신, 최신 위치와 이력의 서버 분리, 기사 운행 전후 확인을 결합한 구조가 적합하다고 판단하였다. 이 결론은 학교 측 운영 제약과 소프트웨어 설계에 근거한 것으로, 실제 단말 설치와 기사 업무 관찰을 통한 검증은 후속 과제다.

제7장 논의

7.1 통합 정보 구조의 의미

UNIBUS의 핵심 결과는 화면 수가 아니라 정보의 단일 출처를 확보한 데 있다. 초기에는 2D, 3D, 관리자 데모가 서로 다른 좌표와 경로를 사용할 수 있었고, 이 경우 관리자가 한 화면에서 정류장을 옮겨도 다른 화면은 바뀌지 않았다. 최종 구조는 `routes`, `route_stops`, `route_shape_points`를 기준으로 모든 표현을 만들고, 시뮬레이션도 같은 `polyline`에서 위치를 계산한다. 이를 통해 데이터 변경과 시각적 표현의 책임을 분리하였다.

학내순환과 신창역 셔틀을 별도 하단 탭으로 완전히 분리하지 않고 셔틀버스 안에서 전환한 결정도 통합의 일부다. 두 서비스는 후문과 일부 차량을 공유하며, 역에서 돌아온 차량이 학내순환을 이어갈 수 있다. 사용자가 하단 내비게이션을 오가며 맥락을 잃는 대신 같은 지도에서 서비스만 전환하도록 하였다. 반면 통학버스는 등하교 시간표와 지역 노선 중심으로 이용 방식이 달라 별도 탭을 유지하였다.

이 구조는 캠퍼스 교통의 다양한 운행 규칙을 무리하게 하나의 시간표 형식으로 축소하지 않는다. `schedule_basis`, 간격, 출발 오프셋, 승차 대기, 연계 노선으로 공통 필드를 두되 각 노선이 필요한 값만 사용한다. 따라서 새로운 철도 시간이나 노선이 추가되더라도 UI에 조건문을 반복하기보다 데이터 규칙을 확장할 수 있다.

7.2 2D와 3D의 상호보완성

2D 지도는 경로 전체와 정확한 좌표를 빠르게 확인하는 데 유리하다. 3D는 후문 구조물, 건물과 도로, 고저차, 버스 진행 방향을 함께 보여주어 발표와 공간 이해에 강점이 있다. 그러나 3D는 초기 다운로드와 GPU 비용이 크고, 작은 모바일 화면에서는 라벨과 차량이 겹치거나 카메라 조작이 어려울 수 있다. 그

러므로 3D를 기본 화면으로 강제하지 않고 사용자가 2D와 전환하도록 한 설계가 적절하다.

Lighthouse 결과에서 독립 3D 화면은 온보딩보다 Performance가 6점 낮고 LCP가 0.751초 길었다. 번들 분석에서도 3D vendor가 가장 큰 체크였다. 이는 기능적 풍부함과 초기 비용 사이의 상충을 보여준다. 현재의 화면 단위 지연 로딩과 precache 제외는 일반 화면을 보호하지만, 3D 진입 자체의 비용을 줄이려면 메시 단순화, 인스턴싱, 텍스처 압축, WebGL 성능 등급별 품질 조절이 추가로 필요하다.

디지털 트윈이라는 표현에도 주의가 필요하다. Liu 등(2023)은 물리-디지털 동기성과 형상 충실도를 평가하였지만 UNIBUS는 실제 센서 연동과 공간 오차 측정이 없다. 따라서 본 결과는 이동 객체와 공간 정보를 통합하는 시각화 인공물이며, 운영형 디지털 트윈으로 발전하기 위한 기반이다. 향후 GIS·BIM 자료를 사용하고 실제 차량 위치 지연과 좌표 오차를 측정할 때 더 강한 디지털 트윈 주장을 할 수 있다.

7.3 ETA의 해석과 확장

현재 ETA는 시연 차량이 경로 위에서 일관되게 움직이고 다음 정류장과 출발시각을 설명하는 데 충분하다. 21개 시험은 이 계산이 경계 시각, 다음 날 전환, 대기 상태, stale 상태에서 정의한 규칙대로 동작함을 보여준다. 그러나 실제 버스 도착시간 연구는 위치 정제, 도로 정합, 실시간·과거 속도, 정차시간, 필터 또는 학습 모델을 사용한다(Cathey & Dailey, 2003; Sun et al., 2007; Han et al., 2020). 현재 값에는 이 자료가 없다.

후속 ETA 개발은 단계적으로 수행하는 것이 타당하다. 1단계는 실제 단말에서 위치를 수집하고 지도 정합 전후의 경로 이탈률과 수신 간격을 측정하는 것이다. 2단계는 정류장 사이의 실제 소요시간과 정차시간 분포를 구축하여 시간대·요일별 경험 평균을 계산한다. 3단계는 결정론적 기준선, 이동평균·Kalman 계열 필터, 통계 모형, LSTM 등의 MAE·RMSE를 같은 검증 기간

에서 비교한다. 열차 연계 노선은 버스 ETA뿐 아니라 열차 지연과 환승 가능성을 별도 상태로 다뤄야 한다.

7.4 기사 조작 제약과 운영 구조

기사의 직접 조작이 어렵다는 학교 측 답변은 단순한 사용성 문제가 아니라 안전과 책임의 문제다. 운행 중 휴대전화 입력을 요구하면 앱 기능이 많아질수록 실제 도입 가능성이 낮아진다. 본 연구는 기사 역할을 제거하지 않고 입력 시점을 운행 전후로 제한하고, 위치는 차량 단말 또는 관제 장치가 자동 송신하도록 책임을 재배치하였다.

이 구조에서 관리자는 차량·기사·노선의 배정과 시간표를 사전에 확정한다. 기사는 정차 중 배정을 확인하고 운행을 시작하며, 단말은 위치와 방향을 자동 전송한다. 서버는 현재 배정과 활성 운행을 확인하고 최신 위치와 이력을 저장한다. 사용자는 기사 입력 여부가 아니라 서버가 확인한 최신 시각을 기준으로 위치 신선도를 판단한다. 운영 종료 후 기사는 종료를 확인하거나 관리자가 비정상 운행을 정리할 수 있다.

실제 적용 전에는 차량 단말의 전원, 네트워크 재연결, 위치 권한, 백그라운드 실행 제한, 단말 분실과 교체, 기사 교대 절차를 검증해야 한다. 자동 송신이 실패할 때 기사에게 운전 중 복구를 요구하지 않고 관리자에게 경보를 보내는 예외 흐름도 필요하다.

7.5 보안과 개인정보 논의

RLS, 역할 미들웨어, 토큰 해시, CORS, 위치 RPC 권한 분리는 브라우저 앱에서 관리 기능을 보호하는 기본 계층을 형성한다. 비인증 시험에서 사용자 관리와 노선 쓰기가 거부되었고 anon 직접 조회에서 사용자·토큰 행이 노출되지 않았다. 하지만 이러한 시험은 정해진 요청 몇 개를 확인한 것이며 전체 침투시험이 아니다.

가장 중요한 개선은 인증 세션을 자바스크립트 접근 영역에서 분리하는 것이다. OWASP 권고와 비교하면 30일 localStorage bearer token은 XSS 발생 시

탈취될 수 있다. BFF 또는 Edge Function이 HttpOnly · Secure · SameSite 쿠키를 설정하고 짧은 세션을 회전하도록 바꾸면 노출면을 줄일 수 있다. 로그인 시도 제한은 계정과 IP, 장치 신호를 함께 사용하고 실패 로그와 관리자 경보를 추가해야 한다.

위치 자료는 차량 위치라도 기사와 운행시간을 결합하면 개인의 근무 정보로 이어질 수 있다. 위치 이력은 목적과 보존 기간을 명시하고 최소한으로 수집해야 한다. 현재 30일 정리 규칙은 기술적 기본값이며 실제 정책은 학교 개인정보 담당자와 협의해야 한다. 연구용 데이터셋을 만들 때는 기사 이름과 계정을 제거하고 운행 ID를 가명화해야 한다.

7.6 연구의 한계

첫째, 학생 · 교수 · 관리자 · 기사 대상 현장 사용성 평가가 없다. 화면 검토와 기능 시나리오는 수행했지만 과업 성공률, SUS, 만족도, 학습시간을 측정하지 못했다. 둘째, 실제 차량 GPS 이력이 없어 ETA의 MAE · RMSE와 위치 오차를 평가하지 못했다. 셋째, 3D 캠퍼스는 수작업으로 단순화한 모델이며 건물 · 도로 · 지형의 측량 정확도를 검증하지 않았다.

넷째, Lighthouse는 공개 경로 두 개와 한 하드웨어 환경에서 수행한 시뮬레이션 측정이다. 로그인 이후 API 호출, Realtime 구독, 저사양 Android, iOS PWA, 캠퍼스 Wi-Fi와 LTE의 차이를 포함하지 않는다. 다섯째, 보안 검토는 자동 점검과 제한된 비인증 요청으로 구성되며 전문 침투시험과 운영 조직 권한은 포함하지 않는다. 여섯째, 열차 시간표 자료의 자동 갱신과 지연 반영이 완성된 운영 인터페이스로 검증되지 않았다.

일곱째, 팀 역할은 구성원이 합의한 업무 분장을 바탕으로 기술했지만 Git 커밋 수로 개인 기여를 정량화하지 않았다. 계정과 이메일, 공동 작업, 병합 방식이 커밋 통계를 왜곡할 수 있기 때문이다. 여덟째, 연구 평가 기준일 이후 개발이 계속될 수 있으므로 결과는 2026년 7월 15일의 코드와 원격 스키마에 한정된다.

7.7 후속 연구와 구현 로드맵

첫 번째 우선순위는 실제 위치 수집의 최소 실증이다. 운행 차량 한 대 또는 비운행 테스트 차량에 고정 단말을 설치하여 1-2주 동안 수신 간격, 누락률, 지도 정합 오차, 배터리와 데이터 사용량을 측정한다. 기사에게 운전 중 조작을 요구하지 않고 연구자가 사전에 단말을 설정해야 한다. 이 자료로 최신 위치의 45초 기준과 30초 이력 표본이 적절한지 검증할 수 있다.

두 번째는 사용자 평가다. 학술제에서 학생과 교수, 심사위원에게 학내순환 찾기, 신창역 출발 확인, 정류장 위치 찾기, 관리자 경로 수정 같은 과업을 제시하고 성공률, 완료시간, 오류, 이해도, SUS를 수집할 수 있다. 이는 실제 운행 효과가 아니라 인터페이스 사용성을 평가하는 연구로 명확히 구분해야 한다. 참여 동의와 익명화, 최소 개인정보 수집 절차가 필요하다.

세 번째는 성능과 보안 개선이다. 3D vendor의 추가 분할, 저사양 품질 프리셋, 모델 인스턴싱, 앱 진입 후 유틸 시간 prefetch를 비교한다. 인증은 HttpOnly 쿠키와 단기 회전 세션으로 전환하고 rate limiting, CSP, 로그 비식별화, Deno check CI를 적용한다. Vercel과 Supabase의 운영 모니터링에서 5xx, API 지연, Realtime 연결, 푸시 실패를 관찰해야 한다.

네 번째는 공간과 예측 고도화다. 학교 GIS 또는 BIM 자료를 확보해 건물 · 도로 · 정류장 좌표 오차를 측정하고, 관리자 편집 결과를 버전 관리하여 잘못된 경로를 되돌릴 수 있게 한다. GPS 이력이 축적되면 구간별 경험 속도와 정차 분포를 기준선으로 만들고 통계 · 학습 모델을 검증한다. GTFS 또는 유사한 정적 · 실시간 feed를 제공하면 향후 다른 캠퍼스 서비스와의 연동 가능성도 높아진다.

제8장 결론

본 연구는 순천향대학교의 학내순환, 신창역 연계 셔틀, 통학버스를 사용자 · 관리자 · 기사 역할과 함께 통합하는 2D/3D 웹 시스템 UNIBUS를 설계하고 구현하였다. 노선, 정류장, 경로 보정점, 시간표 기준, 운행 단계, 차량 상태를 관계형 모델로 구성하고 관리자가 수정한 데이터를 사용자 2D 지도와 3D 캠퍼스가 공동으로 사용하도록 연결하였다. 학내순환 10분 간격, 열차 출발 10분 전 후문발, 열차 도착 5분 후 역발, 후문 도착 후 학내순환 연계라는 운행 차이를 데이터 규칙으로 표현하였다.

현장 운행 시험이 불가능한 조건에서 설계과학 연구의 프로토타입 평가를 적용하였다. TypeScript와 ESLint 검사, production build, 21개 알고리즘 시나리오, 원격 마이그레이션과 DB lint, 비인증 API와 anon RLS 시험, Lighthouse 5회 반복 측정을 수행하였다. 기능 로직과 기본 권한 경계는 정의한 시나리오에서 동작했지만 3D 번들 비용과 LCP, localStorage 세션, 로그인 시도 제한, Deno 타입검사 공백이 남아 있음을 확인하였다. 초기 시드의 고정 자격증명은 현재 코드에서 제거했으나 원격 계정 회전은 운영 확인이 필요하다.

연구질문 1에 대해 공통 노선 · 운행 모델과 역할별 서비스 계층으로 세 교통 유형을 통합할 수 있음을 보였다. 연구질문 2에 대해서는 시뮬레이션과 2D · 3D 표현이 소프트웨어 시나리오 수준의 기능 기준을 충족했으나 실제 ETA 정확도와 사용성은 확인하지 못했다. 연구질문 3에 대해서는 운전 중 기사 입력을 최소화하고 관리자 사전 배정, 자동 위치 단말, 서버 권한 검증을 결합한 구조를 제시하였다.

UNIBUS는 완성된 운영 시스템보다 검증 가능한 통합 프로토타입에 가깝다. 그럼에도 실제 학교의 복합 셔틀 규칙과 역할 제약을 데이터 구조와 실행 화면으로 구체화하고, 구현 결과와 한계를 수치로 함께 제시했다는 점에 의의가 있다. 후속 연구에서 실제 위치 수집, 사용자 과업 평가, GIS 기반 공간 보정,

ETA 모델 비교, 인증 구조 개선을 수행하면 학술제 시연을 넘어 캠퍼스 교통 운영을 지원하는 시스템으로 발전할 수 있다.

참고문헌

- Cathey, F. W., & Dailey, D. J. (2003). A prescription for transit arrival/departure prediction using automatic vehicle location data. *Transportation Research Part C: Emerging Technologies*, 11(3-4), 241-264. [https://doi.org/10.1016/S0968-090X\(03\)00023-8](https://doi.org/10.1016/S0968-090X(03)00023-8)
- Chen, K.-C., Chang, Y.-T., & Hsieh, S.-H. (2024). A digital twin platform based on 3D building models and smart IoT for a climate-resilient campus: A case study of National Taiwan University. In *Computing in Civil Engineering 2023* (pp. 553-561). <https://doi.org/10.1061/9780784485231.066>
- Dailey, D. J., Maclean, S. D., Cathey, F. W., & Wall, Z. R. (2001). Transit vehicle arrival prediction: Algorithm and large-scale implementation. *Transportation Research Record*, 1771(1), 46-51. <https://doi.org/10.3141/1771-06>
- Elliott, T., & Lumley, T. (2020). Modelling the travel time of transit vehicles in real-time through a GTFS-based road network using GPS vehicle locations. *Australian & New Zealand Journal of Statistics*, 62(2), 153-167. <https://doi.org/10.1111/anzs.12294>
- Gonzales, A. T., Thamadharan, K., & Jothi, N. (2024). Efficient campus shuttle tracking and management mobile application for college campus. *Proceedings of International Conference on Artificial Life and Robotics*, 29, 482-486. <https://doi.org/10.5954/ICAROB.2024.OS18-1>
- Han, Q., Liu, K., Zeng, L., He, G., Ye, L., & Li, F. (2020). A bus arrival time prediction method based on position calibration and LSTM. *IEEE Access*, 8, 42372-42383. <https://doi.org/10.1109/ACCESS.2020.2976574>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>
- Hu, P. (2023). Investigation on smart campus management platform based on digital twin. *Procedia Computer Science*, 228, 937-945. <https://doi.org/10.1016/j.procs.2023.11.123>
- Lin, W.-H., & Zeng, J. (1999). Experimental study of real-time bus arrival time prediction with GPS data. *Transportation Research Record*, 1666(1), 101-109. <https://doi.org/10.3141/1666-12>
- Liu, Y., Tu, X., Chen, D., Han, K., Altintas, O., Wang, H., & Xie, J. (2023). Visualization of mobility digital twin: Framework design, case study, and future challenges. In *2023 IEEE 20th International Conference on Mobile Ad Hoc and Smart Systems* (pp. 170-177). <https://doi.org/10.1109/MASS58611.2023.00028>
- MobilityData. (n.d.). *GTFS Realtime best practices*. Retrieved July 15, 2026, from <https://gtfs.org/documentation/realtime/realtime-best-practices/>

- National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B-4)*.
<https://doi.org/10.6028/NIST.SP.800-63B-4>
- NAVER Cloud Platform. (n.d.). *NAVER 지도 API v3 기술문서*. Retrieved July 15, 2026, from <https://navermaps.github.io/maps.js.ncp/docs/>
- Newson, P., & Krumm, J. (2009). Hidden Markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems* (pp. 336-343).
<https://doi.org/10.1145/1653771.1653818>
- OWASP Foundation. (n.d.). *HTML5 security cheat sheet*. Retrieved July 15, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>
- PostgreSQL Global Development Group. (2026). *PostgreSQL 17 documentation: Row security policies*. <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
- Sun, D., Luo, H., Fu, L., Liu, W., Liao, X., & Zhao, M. (2007). Predicting bus arrival time on the basis of Global Positioning System data. *Transportation Research Record*, 2034(1), 62-72. <https://doi.org/10.3141/2034-08>
- Supabase. (n.d.). *Row level security*. Retrieved July 15, 2026, from <https://supabase.com/docs/guides/database/postgres/row-level-security>
- World Wide Web Consortium. (2026a). *Service Workers Nightly*.
<https://www.w3.org/TR/service-workers/>
- World Wide Web Consortium. (2026b). *Web Application Manifest: W3C Working Draft 7 May 2026*. <https://www.w3.org/TR/appmanifest/>
- Xu, H., Shao, Y., Chen, J., Wang, C. R., & Berres, A. (2024). Semi-automatic geographic information system framework for creating photo-realistic digital twin cities to support autonomous driving research. *Transportation Research Record*, 2678(6), 1068-1084.
<https://doi.org/10.1177/03611981231205884>

부록 A. 상세 시험 항목

표 A-1. 알고리즘 시험 21개 항목

ID	영역	검증 내용	결과
SCH-01	시간표	유효한 시각만 정렬	통과
SCH-02	시간표	학내순환 10분 간격	통과
SCH-03	시간표	경계 직후 다음 간격	통과
SCH-04	시간표	열차 출발 10분 전	통과
SCH-05	시간표	열차 도착 5분 후	통과
SCH-06	시간표	다음 날 첫 운행	통과
SCH-07	시간표	학내순환 연계 설명	통과
MOT-01	경로	위도 0.001도 거리	통과
MOT-02	경로	0.5 m 중복점 제거	통과
MOT-03	경로	지정 간격 재표본화	통과
MOT-04	경로	절반 지점 표본	통과
MOT-05	경로	정북 방위각	통과
ETA-01	ETA	최신 위치의 다음 정류장	통과
ETA-02	ETA	45초 stale 판정	통과
ETA-03	ETA	운행 없음 대기 상태	통과
ETA-04	ETA	역 대기 계획 출발 반영	통과
SIM-01	시물레이션	학내순환 3대 생성	통과
SIM-02	시물레이션	동일 입력 결정성	통과
SIM-03	시물레이션	24시간 캠퍼스 좌표 경계	통과
SIM-04	시물레이션	신창역 왕복 · 양 끝 출발	통과
SIM-05	시물레이션	24시간 역 셔틀 좌표 경계	통과

부록 B. 팀 역할과 개발 일정

표 B-1. 팀 구성과 담당 역할

구성원	담당	논문에 반영된 주요 책임
권재원 (20214157)	PM, 프론트엔드 · 백엔드 총괄	요구사항, UI/3D, API 통합, 배포 · 평가 총괄
박소정	백엔드	서버 기능과 업무 API 개발
손성윤	프론트엔드	사용자 화면과 컴포넌트 개발
김동하	데이터베이스	스키마와 데이터 구조 지원
정영신	백엔드	서버 기능 개발 지원
강민영	백엔드	서버 기능 개발 지원

표 B-2. 개발 및 연구 일정

기간	활동	결과
2025-09-09 이후	아이디어 수집 · 주제 탐색	캠퍼스 셔틀 통합 주제 선정
2026-03-02 이후	본격 개발	사용자 · 관리자 · 기사, DB · API 구현
2026-06-07	경로 · 배차 · 보안 개선	관리자 편집, 2D/3D, PWA 확장
2026-07-15	논문용 정량 평가	시험 · DB/API · Lighthouse · 보안 결과
이후	학술제 시연 · 보완 예정	사용자 평가와 실제 위치 실증 후보

부록 C. 연구윤리 및 AI 활용 고지

본 연구는 실제 학생 · 기사의 개인정보나 위치 이력을 수집하는 현장 실험을 수행하지 않았다. 화면 캡처에는 토큰, API 키, 개인 이메일과 학번이 노출되지 않도록 확인하였다. 향후 사용자 평가나 차량 위치 수집을 수행할 경우 참여 동의, 연구 목적과 보존 기간 고지, 가명화, 접근권한, 철회 절차를 별도로 마련해야 한다.

연구와 원고 작성 과정에서 AI 기반 연구 보조 도구를 문헌검색, DOI·출처 교차검증, 코드 구조 분석, 시험 설계, 도표와 원고 초안 작성에 사용하였다. 모든 수치 결과는 저장소의 명령 실행 또는 연결된 원격 데이터베이스에서 다시 확인했고, 문헌 주장은 DOI 원문·출판사 초록·공식 표준 문서와 대조하였다. AI가 실제 사용자 시험이나 측정 결과를 생성한 것처럼 서술하지 않았으며, 최종 연구 책임과 검토는 저자에게 있다.

Abstract

Design and Implementation of an Integrated University Shuttle Bus Information System Based on a 2D/3D Digital Twin: A Prototype for Soonchunhyang University

University shuttle services often combine campus loops, railway-station connections, and regional commuter routes with different scheduling rules. When route geometry, stop locations, vehicle states, notices, and role-specific operations are maintained separately, passengers cannot obtain a consistent view of service availability and administrators must repeat the same change across multiple interfaces. This study designed and implemented UNIBUS, an integrated shuttle information system that combines a two-dimensional web map with a three-dimensional campus representation for the transportation context of Soonchunhyang University.

The study followed a design science research process consisting of problem identification, objective definition, artifact design and development, scenario demonstration, and quantitative evaluation. The artifact uses a React and Vite progressive web application, the NAVER Maps API, React Three Fiber, Supabase Edge Functions, PostgreSQL, and Realtime. Its route model represents a 10-minute campus-loop interval, a departure from the rear gate 10 minutes before a train leaves Sinchang Station, and a departure from the station approximately 5 minutes after a train arrives. It also distinguishes a station-to-campus trip that terminates at the rear gate from one that continues for a single campus loop. User, administrator, and driver roles are separated across routes, APIs, and database policies. Because the university indicated that direct driver interaction while driving was not operationally feasible, the proposed architecture minimizes in-motion input and assumes automatic location transmission by a vehicle-mounted terminal.

Evaluation focused on prototype validity rather than field effectiveness. TypeScript and ESLint checks completed with zero errors, and all 21 deterministic scenarios for scheduling, route motion, ETA states, and simulation passed. Eighteen local migration versions matched the remote Supabase history, and linked database lint reported no schema errors. Unauthenticated requests to user administration, driver status, and route creation endpoints returned HTTP 401, while public route retrieval remained available. No known dependency vulnerabilities were reported by npm audit. Across five Lighthouse runs under the default mobile simulation, median Performance scores were 79 for onboarding and 73 for the

independent 3D page. Both routes scored 100 for Accessibility and Best Practices with zero Total Blocking Time and Cumulative Layout Shift. However, the largest 3D vendor chunk was 256.47 kB gzip and the median 3D Largest Contentful Paint was 4.654 seconds, indicating remaining optimization work.

The study contributes a unified route and service model that connects administrator-edited stops and paths to consistent 2D and 3D passenger views. Its results do not establish real-world arrival prediction accuracy, usability, operational efficiency, or safety. The displayed ETA is a deterministic estimate based on route progress and configured speed because no historical GPS, traffic, or dwell-time data were available. Field location collection, task-based usability testing, GIS-based spatial validation, predictive model comparison, and session-security improvements are required before operational deployment.

Keywords: university shuttle bus, digital twin, real-time location, arrival time estimation, 3D campus, progressive web application, Supabase, design science